

HYPocompass: Large-Language-Model-based Tutor for Hypothesis Construction in Debugging for Novices

Qianou Ma
qianouma@cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Kenneth Koedinger
koedinger@cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

Hua Shen
huashen@umich.edu
University of Michigan
Ann Arbor, MI, USA

Tongshuang Wu
sherryw@cs.cmu.edu
Carnegie Mellon University
Pittsburgh, PA, USA

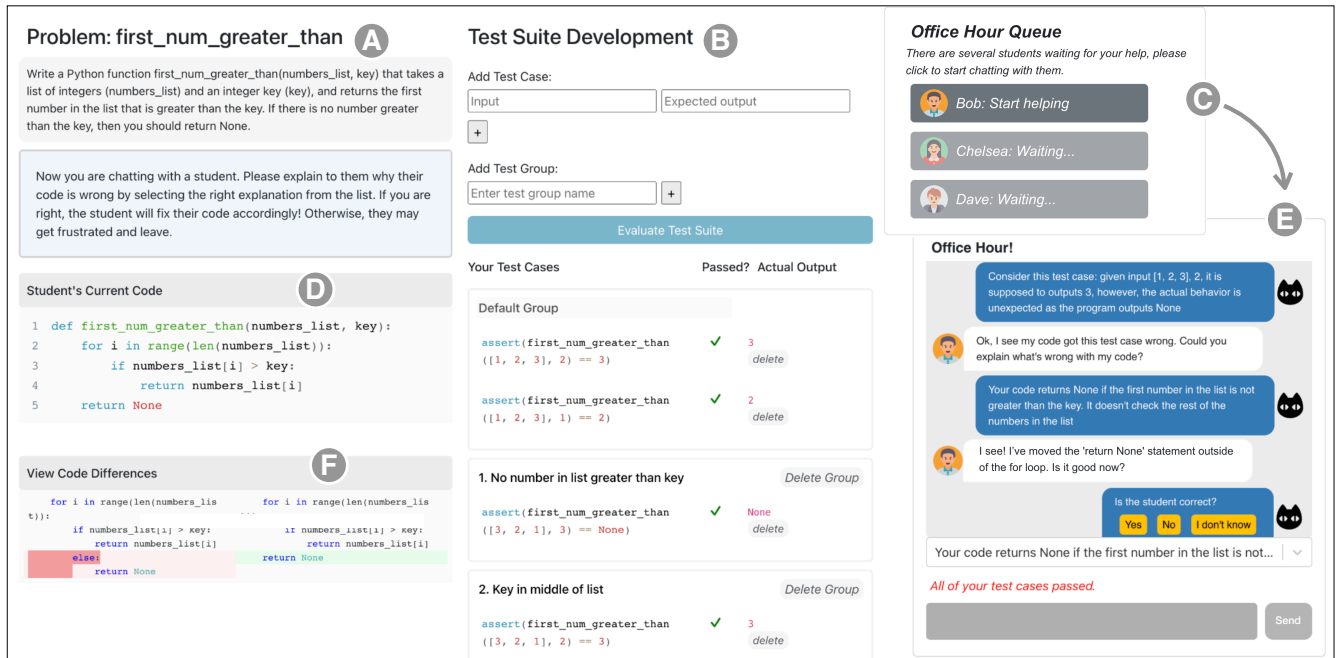


Figure 1: The HYPocompass interface provides students with end-to-end deliberate practice on hypothesis construction in debugging. Given a programming exercise description (A), a student user (in the role of a Teaching Assistant) needs to first compile a test suite for auto-grading (B), then assist multiple LLM-simulated student agents in an Office Hour Queue (C). Each LLM-agent acts as a novice seeking help with a buggy solution (D). The student user will assist the agent through a dialog interface (E), generating hypotheses on why the code malfunctions, writing additional test cases that reflect their hypotheses, and providing natural language explanations. The LLM agent will provide immediate feedback to the user based on the correctness of their explanations, either by making the right local fixes to the code (F) or by sending messages expressing confusion about the explanation.

ABSTRACT

With the prevalence of imperfect but capable LLMs in software development, it becomes increasingly important for developers to cultivate debugging skills — to form hypotheses about the source of error in both their own codes and codes produced by their AI pair-programmers. Despite the necessity, hypothesis construction in debugging is rarely taught due to a lack of explicit instruction. In this work, we explore whether LLMs can be used to train novices on hypothesis construction, by designing a theoretically motivated,

LLM-augmented tutor — HYPocompass. HYPocompass relies on LLMs for generating rich training materials guided by learning principles and presents them in a learning-by-teaching environment, where LLMs act as students who write bugs and attempt to fix them, and human novices focus on debugging in the role of a Teaching Assistant. Evaluations show that HYPocompass consistently generates high-quality training materials, and brings significant learning gain: In a pre-to-post test setup, 10 novices improved their performances by 17%, with a reduced completion time of 13%.

CCS CONCEPTS

- **Human-centered computing** → **Interactive systems and tools**; • **Applied computing** → **Computer-assisted instruction**;
- **Computing methodologies** → **Natural language generation**.

KEYWORDS

LLM, teachable agent, hypothesis construction, debugging, CS1

1 INTRODUCTION

Recently, Large Language Models (LLMs) have become a widespread part of everyday software development. It is reported that users of GitHub Copilot (an LLM-based, commercialized programming assistance service) rely on it to produce as much as 46% of their code [23]. In fact, Copilot is advertised as “your AI pair programmer” [18]. LLMs are versatile to support various needs of programmers, including automatically completing code [4, 8], auto-generating test cases [51], and providing natural language interaction and feedback for code fixing [28, 75, 97]. The widespread availability and flexibility of LLMs provide clear evidence that coding in conjunction with LLMs will *increasingly become the norm*.

Meanwhile, these models still tend to exhibit unpredictable performance [36] and instability [40]. Even GPT-4, the most advanced LLM to date, still makes mistakes 17% of the time in coding tasks for introductory and intermediate programming courses [80]. As a result of these model deficiencies, when programmers delegate code writing to LLMs, they are also “forced” to spend a significant amount of time thinking and verifying LLM suggestions [69]. Even so, programmers still tend to overly rely on LLM generations and incorrectly treat these models as oracles [79]. Educators express particular concern about novice students falling prey to these confirmation biases since they are less prepared to critically assess the generated code [11, 12, 31]. The tension between the human-LLM co-programming popularity and the risk associated with overreliance gives rise to the necessity of training developers in debugging, testing, and reviewing code skills [11, 12, 31].

Despite the importance of debugging, in practice, there is still a lack of explicit debugging instruction at schools [1]. In an interview conducted by Perscheid et al. [73], none of the professional developers recalled receiving formal instructions on debugging. Several factors contribute to the limited teaching of debugging (especially in CS1, the introductory Computer Science classes [41]), such as a shortage of time and resources to create tailored debugging materials and assessments [65]. As a result, students mostly learn from their own mistakes when solving problems. This can potentially involve a lot of frustrations, as students need to spend a great amount of time and effort finding bugs, together with all other cognitively demanding tasks of understanding, tracing, and writing code [34].

These challenges prompt us to ask: **Can we train students to better debug code written by themselves and their AI pair programmers, by providing explicit and scaffolded debugging instructions with minimal costs to instructor time?** We are particularly interested in utilizing LLMs to facilitate debugging skill training, as they have the potential to support: (1) **Guided debugging on LLM generations**: The new norm of programming with AI urges us to prepare our students for debugging AI-generated codes [59]; however, students struggle to debug LLM generations

with existing tools [11]. By presenting LLM bugs in a training environment, we can simulate a more supported debugging experience with LLMs for students; (2) **Explicit and scaffolded instruction**: Leveraging the functional versatility of LLMs in code generation and bug fixing, we can develop scaffolded instructions to help students explicitly practice hypothesizing about code defects, while offloading some of the cognitively demanding aspects of debugging to LLMs (e.g., code completion); (3) **Tailored and efficient instruction development**: LLMs can easily generate large amounts of text including test cases [51], bugs [5], bug explanations and hints [74], with reasonable performance if carefully prompted. Therefore, it is possible to automate the creation of *tailored* instructional materials, thereby providing deliberate debugging practice while requiring minimal instructor effort. However, with the aforementioned risk of overreliance, it remains challenging to develop an effective and engaging learning environment where we can take advantage of LLMs without hindering student learning [2].

In this work, we seek to instantiate *the practical use of LLMs* in debugging training with HYPOCOMPASS (Figure 1). HYPOCOMPASS is an **interactive, LLM-augmented intelligent tutoring system that explicitly trains students in hypothesis construction**, i.e., forming hypotheses about the source of error — a major roadblock to students’ debugging success [32, 34]. To achieve the aforementioned three potential benefits, we put substantial effort into designing two primary building blocks:

First, we develop an interaction mechanism inspired by established learning theories, for **explicit and engaging learning in an error-tolerant environment** [47, 91]. As shown in Figure 1, we adopt the teachable agent framework [13]: we have the LLM-agent play the role of students who wrote diverse buggy solutions to a given programming problem, such that human novices (i.e., system users) act as Teaching Assistants (TAs) to guide the LLM-agent through the debugging process. We establish clear task delegation, with users engaging in TA-like tasks related to hypothesis construction (e.g., writing test cases in Figure 2C), while adjacent tasks are handled by the LLM (e.g., bug-fixing). This learning-by-teaching setup not only creates an environment where *model errors serve as valuable learning opportunities*, but it can also reduce frustration and enhance learning [13, 64, 71].

Second, we create a reliable content development pipeline using LLMs to **generate customized instructional materials for deliberate practice**. We start by over-generating practice suites with GPT-3.5/GPT-4 necessary for the HYPOCOMPASS interaction, including buggy code, explanations, fixes, and various hints. We ensure quality through task design and manual verification. Next, we select practice problems using automatic algorithms, prioritizing potential learning gains. This approach enables minimal efforts of customization for different programming problems and students.

To eventually deploy HYPOCOMPASS in CS1 classrooms, we conducted in-lab evaluations on both of its building blocks. To assess the quality of the LLM-generated training materials, we performed manual inspections on 6 practice problems and 145 buggy codes. We found that HYPOCOMPASS can *stably generate and tailor training materials with minimal additional instructor effort*: A complete set of practice materials with 9 buggy codes, explanations, fixes, 10 test case hints, and 3 test category hints can be generated with a 90% success rate and only take 15 minutes to label and edit. Though we

currently pre-generate all the materials for safety reasons, the high success rate demonstrates HYPOCOMPASS’s potential to facilitate real-time interactions between LLM agents and users.

Then, to assess the effectiveness of HYPOCOMPASS in facilitating learning, we conducted a user study with 12 participants. Using pre- and post- tests carefully designed in alignment with our learning objectives of hypothesis construction, we showed that the performance of novice students improved significantly by 17%, and completion time was reduced by 13% after utilizing HYPOCOMPASS. Discussions with students further highlighted that HYPOCOMPASS motivates more systematic hypothesis testing, well-scaffolds the learning experience, and boosts students’ engagement and confidence in debugging.

We conclude the paper by discussing key factors to the success of HYPOCOMPASS: identifying the right task formation for LLMs, training humans on skills complementary to LLM counterparts, and creating environments where humans can actively interact with and take advantage of LLM mistakes. In summary, we contribute:

- Theoretically motivated instructional design for training students’ hypothesis construction skills. We carefully identify learning objectives, implement HYPOCOMPASS according to established cognitive debugging process and other learning principles, and design learning assessments that are well-aligned with the learning objectives. To the best of our knowledge, we are the first to provide aligned instruction and assessments on the hypothesis construction learning objectives.
- The implementation of HYPOCOMPASS, which consists of multiple components that can be easily adapted and re-used. For example, our carefully designed prompt, task formation, and selection algorithms make it possible to use GPT-3.5/GPT-4 to generate materials for deliberate practice in debugging, in a way that maximizes performance and minimizes risks.
- An exploration of employing LLMs to enhance learning. We identify and utilize the potential benefits of LLMs, especially in material generation and skill offloading. We further create an environment where model mistakes can be taken as students’ learning opportunities, which makes the use of imperfect LLMs practical and feasible.

2 BACKGROUND: CURRENT DEBUGGING INSTRUCTION

We conducted a literature survey and a formative study to gain insights into current practices, challenges, and existing gaps in debugging instruction. As illustrated below, we discovered a lack of explicit instruction and assessment on comprehensive and accurate hypothesis construction. In particular, instructors struggle to find materials that are engaging and easy to adapt to CS1 classes.

2.1 Formative Study: Debugging Instruction and Challenges in Practice

While many prior studies flag that teaching debugging is challenging and is rarely included in curricula [65, 73], to the best of our knowledge, there is still a lack of in-depth analysis as of *why debugging training is challenging in practice* (though some ad-hoc online discussions do exist [1]). To address this gap, we conducted an online survey with practitioners in online computer science teaching

communities. We collected and analyzed responses from 10 instructors and one CS education researcher coming from 9 different R1 universities. These educators cover a range of subjects including CS1 (6), CS2 (4), system (3), software engineering (2), data science (1), and other CS areas. We posed three key questions:

- Q.1 Is debugging and testing explicitly taught in your school/class (or from what you have experienced)? If yes, how was the experience?
 Q.2 If not, what is the roadblock?
 Q.3 Any general comments on debugging instructions at schools?

For Q1, participant reflected on the *status-quo* at their universities, encompassing various instructional formats (e.g., lecturing, live coding demonstrations, and mentoring during office hours), assignment types (e.g., lab work), and assessment methods (e.g., quizzes, exams, and homework with auto-graders). For Q2, participants discussed challenges related to time constraints, curriculum and assessment design complexity, student motivation, and instructor motivation. For Q3, participants commented that debugging instruction is “*extremely needed*” (F2), “*I know our students want to learn [debugging] better*” (F6), and “*my [debugging tutorial] continues to be my highest viewed video (over quarter million views), which tells me that yes, there are general deficiencies in many curricula with respect to debugging*” (F7).

In the following sections, we summarize some aspects of the survey response together with the literature. There are more details that are out of the paper’s scope and saved for future analyses.

2.2 Opportunities for Improving Debugging Instruction

2.2.1 Debugging needs targeted instruction: The importance of hypothesis construction.

Existing literature indicates that the primary challenges of debugging for novices lie in program comprehension (i.e., understanding the code) and finding bugs [65]. Once bugs are identified, fixing bugs is perceived to be easier in comparison [34]. While many studies have attempted to improve students’ program comprehension via code-tracing and other instructional activities [44, 58, 89], there has been less emphasis on instruction specifically targeting bug finding — the few tutoring systems or instruction [7, 16, 48, 57, 84] tend to be limited in terms of the programming languages they covered and the scope of their training objectives. In particular, though prior work confirms that students’ debugging skills can be improved with explicit instructions aligned with the cognitive model of systematic debugging [47, 67], few studies designed instructions accordingly. Therefore, we need to *anchor the instruction on finding bugs in the cognitive process of finding and hypothesizing about bugs*.

Araki et al. [6], Xu and Rajlich [91] characterized the cognitive model of locating and correcting errors in programs as a **hypothesis construction and verification process**. This process entails iterative steps, including initializing the hypothesis set, modifying the hypothesis set, selecting a hypothesis, and verifying the hypothesis (Figure 2B). In the context of programming, Zeller [95] described this process as “scientific debugging,” and it can be instantiated as *test suite construction*, where different types of test cases represent various hypotheses [84].

This hypothesis-driven debugging approach has been taught both by our formative study instructors (F3 & F7) and in the literature [84]. Prior works found that novices struggle to generate comprehensive hypotheses, as they have difficulty writing test cases that can reveal hidden bugs [25, 26]. Novices also encounter challenges in using hypotheses to locate and fix bugs — they tend to work narrowly with a single hypothesis regarding the bug’s location or simply make random changes without a specific hypothesis in mind [34, 84, 85]. In contrast, experts can systematically generate and explore hypotheses about bugs in code [25, 26]. These differences underscore the importance of teaching students to **construct accurate hypotheses about bugs and develop comprehensive hypotheses regarding potential bugs in a program**.

2.2.2 Debugging needs explicit instruction: Motivating and scaffolding students for deliberate practice.

When reflecting on debugging training challenges, three participants cited the lack of student motivation as a primary challenge. For example, instructor F7 commented, “[Students] don’t have enough experience or complex enough programs to make debugging worth it and so consequently don’t see the need/use for it.” Prior literature also found that students tend to appreciate the importance of code testing only when it’s relevant to their own tasks [43]. However, novice students may not be able to learn enough just by debugging their own codes. For example, F8 recalled repeatedly hearing from students saying: “I was taking test, I made a tiny mistake and I couldn’t fix it.” Moreover, when students learn debugging in-the-wild, they are often involved in immediate problem-solving and need to implicitly keep all the hypotheses and outcomes in memory [95], which leads to further frustration and a loss of motivation [34].

As such, to motivate students, **it is crucial to explicitly teach debugging with sufficient scaffolding and in contexts relevant to students**. A natural option would be to teach it as part of the CS1 class. Unfortunately, one major roadblock for providing explicit debugging instructions is the packed curriculum: “there is already so much curriculum to get through, causing issues for where/how to fit [debugging instruction] in and assess it” (F10). Creating scaffolding materials is also time-consuming, for example, instructor F9 has integrated debugging support into assignments by manually writing customized feedback and hints on test cases for students, “but annotating those ourselves took FOREVER”.

As an alternative, it seems promising to provide instructions using debugging tutors, which is the focus of our work. These tutors offer immediate feedback and interactivity outside of class, requiring minimal instructor efforts. Automatic systems can provide engaging instruction, e.g., Ko and Myers [48] mentioned that users find Whyline to be enjoyable, like a treasure hunt.

2.2.3 Debugging needs tailored instructions: Generating high-quality materials efficiently.

Six of the instructors in our formative study have experimented with making debugging instructional materials for students to practice and corresponding assessments; they attempted to design labs and assignments with scaffolding to allow students to practice testing and debugging strategies using their own code. However, this can be very time-consuming, as commented by F4, “we’ve had years of trying to build a debugging lab that seems to work.” In fact, according to the literature, integrating test-driven development as debugging

training into CS1/CS2 curricula can incur a setup cost of nearly 200 hours for modifying labs, projects, and grading criteria. Additionally, it can significantly increase students’ workload, leading to a close to 40% increase in hours spent working on projects [22].

Some debugging tutors [16, 48, 57] have attempted to automate the authoring of exercises, generating buggy code, and providing feedback. However, most of these relevant materials and hints still require significant manual labor or are tightly bound to specific programming languages. For example, Ladebug [57] requires teaching staff to manually include errors and define the lines of buggy code. It is therefore challenging to tailor or adapt content dynamically to different classes. Addressing the challenge of material creation overhead could significantly reduce the burden on instructors, and Raman and Kumar [76] suggests that LLMs may alleviate some of the workload, which is what we explore in this work.

3 LEARNING OBJECTIVES & DESIGN RATIONALE

3.1 Learning Objectives: Comprehensive and Accurate Hypothesis Construction

Grounded in the cognitive model [91] and the novice-expert difference [34] in hypothesis-driven debugging, and motivated by the lack of explicit instruction of hypothesis construction in debugging (Section 2), we specify two crucial learning components for HYPOCOMPASS: comprehensiveness and accuracy. We want students to be able to *construct comprehensive hypotheses about what might go wrong in a program given the problem specification*, and we want them to be able to *construct accurate hypotheses about bugs given a bug-revealing test case*.

In a scientific debugging context, hypothesis construction is closely connected with test suite development. A test case specifies a scenario in which the code may fail, which essentially is an instantiation of a *hypothesis* of what can go wrong in the program. A test suite, on the other hand, is a collection of test cases. Running the test suite against code essentially involves conducting experiments and checking whether the code fails in any scenario [95], mapping to *comprehensive hypotheses*. Therefore, we outline two concrete learning objectives for HYPOCOMPASS in the context of writing test cases. Through practice, students should be able to:

- LO1 **Comprehensive Hypothesis Construction:** construct a more comprehensive test suite efficiently and systematically, given just the problem specification.
- LO2 **Accurate Hypothesis Construction:** explain the bug revealed by a failed test case correctly and efficiently, given a specific buggy code.

3.2 Design Rationale

In response to our formative study and survey on literature, and in accordance to our learning objectives, we specify the following four primary goals for designing HYPOCOMPASS:

- R.1 **Safely and appropriately expose students to LLMs.** In addition to the potential of using LLMs in CS education (Section 1), educators are cautious about the possibility that the incorrect use of AIs increases plagiarism or misleads students (e.g., students treating LLMs as oracles can over-trust generated codes

or adopt inefficient algorithm design [17, 21, 83]). To maximize the benefit and minimize the risk of using LLMs, we aim to build the tutoring system in a context where *LLM mistakes will not be detrimental, but is rather useful to student learning*.

- R.2 **Provide explicit practice on comprehensive and accurate hypothesis construction.** As mentioned in Section 2, debugging involves multiple sub-tasks ranging from understanding the code, finding bugs (hypothesizing about bugs), to fixing bugs, and hypothesis construction is reported to be a primary bottleneck for students [34]. Existing instructions and tools often train students on the full debugging process and focus on improving the accuracy and efficiency of bug fixing [66]. As a result, students may not get enough effective and efficient practice on hypothesis construction since they need to juggle between different sub-tasks in debugging. To enable explicit and deliberate practice on hypothesis construction, we need to disentangle the stages such that students practice on tasks directly relevant to our learning objective, while *other adjacent tasks are offloaded to the tutoring system*.
- R.3 **Enable scaffolded and engaging learning with existing learning principles.** To reduce student frustration when they learn debugging implicitly on their own without feedback and scaffolding [34], we need to carefully follow established learning theories, and design tasks, hints, and immediate feedback that can sufficiently support students when they use our tool to practice themselves. We also need to present the instructions in an engaging and motivating way for students.
- R.4 **Efficiently generate tailored instructional materials with minimal instructor effort.** To ensure that HYPOCOMPASS can be easily incorporated into various CS1 courses with little extra instructor time, we hope to automate the training material generation and make the generation as flexible as possible (e.g., generate the practices end-to-end entirely based on programming exercise descriptions, while being programming-language-agnostic). Moreover, as students may have approached the same practice in different ways [33], being able to tailor the materials (e.g., hints and feedback) is essential.

4 THE DESIGN OF HYPOCOMPASS

4.1 Interface Overview

We build HYPOCOMPASS into an interactive interface as shown in Figure 1. The general interaction is intentionally intuitive: When novice users enter HYPOCOMPASS, they will be asked to play the role of a Teaching Assistant (TA) and be prepared to help multiple fake students (simulated by LLMs, referred to as *LLM-agents*) on their codes. Users will first be tasked to **write a comprehensive test suite** for auto-grading, given a programming exercise description. Taking Figure 1A as an example:

```
Write a Python function first_num_greater_than
(numbers_list, key) that takes a list of integers
(numbers_list) and an integer key (key), and re-
turns the first number in the list that is greater than
the key. If there is no such number, return None.
```

The user constructs the test suite by crafting test cases in the format of paired (input, expected output) in Figure 1B, and **sorts**

them into test categories that represent different *hypotheses* of what inputs may trigger errors in code. For example, test case `assert(first_num_greater_than([3,2,1], 3) == None)` can fall under the test category *No number in list greater than key*. HYPOCOMPASS provides some test categories as starter hints, but the user is encouraged to create more.

Once the user is satisfied with their test suite, HYPOCOMPASS shows them an Office Hour Queue (OHQ) simulator, allowing the user to **start helping a series of fake students** (LLM-agents). As they interact with each LLM-agent (beginning with Bob in Figure 1C), the agent surfaces a code snippet that contains one or multiple bugs (Figure 1D). The user **guides the agent in debugging their code through a dialog interface** (Figure 1E), constructing hypotheses on why the code malfunctions, selecting or creating test cases that reflect their hypotheses of the bug, and selecting correct explanations for the bug among a pool of candidates. Here, the user evaluates the code on the test suite in Figure 1B, and notices that it fails on `assert(first_num_greater_than([1,2,3], 2) == 3)`. The user then explains the bug in natural language (Figure 1E):

“Your code returns None if the first number in the list is not greater than the key. It doesn’t check the rest of the numbers in the list.”

The LLM-agent then **uses the flagged test case and explanation to revise the code**, providing immediate feedback to the user. If the explanation is correct, the agent will provide the user with minimal, local fixes to the code, with the edits clearly color-coded in a Code Diff View Figure 1F (here, the `return None` statement is moved outside of the loop). Otherwise, the LLM-agent will nudge the user to reflect on their hypothesis by responding with a confusion message that highlights the discrepancy between the user’s explanation and the actual code behavior (more details in Figure 3b).

Once the user correctly confirms that all the bugs are fixed, they can **move to help the next LLM-agent** (e.g., Chelsea in Figure 1C). Once they complete the Office Hour Queue, HYPOCOMPASS will provide the next round of exercises with another programming problem. While the numbers are configurable, by default HYPOCOMPASS includes two programming exercises, each with three LLM-agents (buggy codes).

4.2 Components inspired by Learning Theory

Underlying the HYPOCOMPASS interface, we motivate the key interaction mechanisms in accordance with existing learning principles and our design rationales (Section 3.2), detailed below.

4.2.1 Practices following cognitive debugging process model (R.2).

As shown in Figure 2D, in HYPOCOMPASS, users primarily perform two tasks that are grounded by our learning objectives (Section 3.1): (1) making the test suite more complete, which reflects comprehensive hypothesis construction (LO1); and (2) correctly mapping explanations to code bugs, which reflects accurate hypothesis construction (LO2). Furthermore, their interaction flow (Figure 2C) is closely aligned with the established cognitive model for debugging process [91] (Figure 2B). By seamlessly integrating the learning objectives, cognitive models, and interaction designs, we optimize HYPOCOMPASS to effectively train students in honing their hypothesis construction skills.

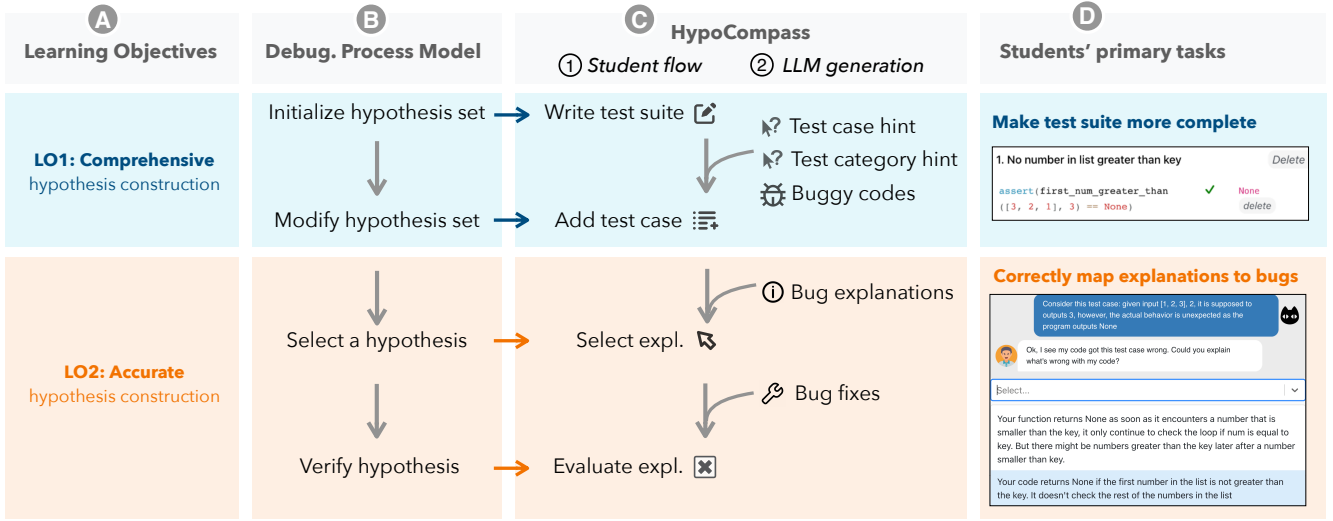


Figure 2: To theoretically ground HypoCompass with existing learning principles, we establish a close mapping between the (A) learning objectives, (B) the cognitive debugging process model [91], (C) the HypoCompass interaction flow, and (D) the primary tasks users perform in HypoCompass. In (C), we further detail how the five types of LLM-generated materials support the student interaction flow.

4.2.2 Teachable agent for engagement (R.3) and error-tolerance (R.1).

Taking advantage of the fact that LLMs can simulate different personas and a tutoring interaction [62], we have the LLM play the role of students who wrote buggy solutions to a given programming problem, such that human students act as TAs to guide the agent through the debugging process. This setup supports learning, as reasoning through diverse bugs helps TAs reinforce their own problem-solving skills [77]; it also boosts students' motivation and protects their self-efficacy as they work through "other people's errors" [13, 64, 71]. Moreover, with LLMs playing the role of students, HypoCompass creates an environment *where model mistakes can be taken as human learning opportunities*, which makes the use of imperfect LLMs practical and feasible, and also actively involves novices in identifying mistakes in LLM-generated code (R.1).

4.2.3 Explicit instruction on hypothesis construction (R.2).

Prior work shows that mastering a new skill requires *deliberate practice* [15, 27], i.e., intentional, targeted, and repetitive exercises with immediate and expert feedback for a period of time. In HypoCompass, we disentangle hypothesis construction from other debugging components in several ways:

- We **minimize the overhead of code comprehension** (which empirically is challenging and time-consuming for novices [66, 89]), by generating training materials using programming problems that students have seen (e.g., in their assignments, quizzes, or class examples) [32].
- We **minimize context-switching to different problems** by using an Office Hour Queue simulation (Figure 1C). Users always assist 2-3 LLM-agents (debug 2-3 codes) before moving to the next programming exercise. This approach exposes them to alternative hypotheses by encountering diverse bugs within the same programming scenario.

- We **offload the necessary code-writing sub-tasks to the LLM**. This involves generating faulty code for a given programming exercise and resolving bugs based on bug explanations. As such, users gain experience in constructing hypotheses without needing to worry about writing syntactically correct code, akin to the task division between developers and their AI pair programmers [59, 69].

4.2.4 Scaffolded learning with hints and immediate feedback (R.3).

HypoCompass provides multiple types of hints (Figure 3a) to support students when they struggle and guardrail students' progress in the tutoring procedure [63]. As in Figure 3a, these hints and feedback include:¹

- **Starter test category hint:** HypoCompass provides three important aspects to test for each programming exercise, displayed as test group headers in Figure 1B. These categories scaffold users when they develop their test suite.
- **Descriptive test case hint:** The initial test suites written by novices tend to be incomplete and, as a result, in many cases, students may run into buggy codes that pass all the existing test cases but fail on some cases that they missed. This can create the illusion that the code is flawless. To assist students in pinpointing the bug (LO2), we offer descriptions of the missing test case, which also helps students create a more comprehensive test suite (LO1).
- **Candidate explanation pool:** We scaffold students with a pool of explanation options to select from, such that they do not have to write it from scratch (which caused frustrations in our pilot). For each bug, the explanation pool contains a *correct explanation*, and two *distractor explanations* (Figure 3a). As explained in

¹Note that many of these can be eventually removed in accordance with the faded scaffolding learning principle [49]. We test some of these in Section 7.

Starter test category hint 1

- 1. No number in list greater than key Delete Group
- 2. Key in middle of list Delete Group
- 3. All numbers in list greater than key Delete Group

Descriptive test case hint 2

Hi Christina! Here is my code and I think I have some problem with it. Can you walk me through that?

Write a test case to cover the scenario where the key number is present in the list, but there are numbers greater than the key

Explanations to choose from 3

Ok, I see my code got this test case wrong. Could you explain what's wrong with my code?

Your code returns None if the first number in the list is not...

Your function returns None as soon as it encounters a number that is smaller than the key, it only continue to check the loop if num is equal to key. But there might be numbers greater than the key later after a number smaller than key.

Your code returns None if the first number in the list is not greater than the key. It doesn't check the rest of the numbers in the list Correct explanation

Your function is printing the number instead of returning it, so it always returns None Distracting explanations

(a) HYPOCOMPASS offers three types of hints to unblock students when they struggle with the next steps, and guardrail their learning procedures: (1) *test category hints*, helping students to brainstorm the test suite comprehensively and systematically; (2) *descriptive test case hints*, describing the input scenario and encouraging students to add test cases they previously missed; (3) *candidate explanation pool*, allowing students to clarify misconceptions without the overhead of writing explanations from scratch.

Feedback to incorrect test cases 1

Add Test Case:

[3, 2, 1], 3 NONE +

The correct expected output is None, please try again.

Feedback to wrong explanation selection 3

But if that's the case, shouldn't first_num_greater_than([1, 2, 3], 2) give me None? Currently my code would output 3.

Your code is currently adding the index of the element to...

Try to select another explanation

Feedback to correct explanation selection 2

I see! I've moved the 'return None' statement outside of the for loop. Is it good now?

Your code returns None if the first number in the list is not...

All of your test cases passed.

View Code Differences

```

for i in range(len(numbers_list)):
    if numbers_list[i] > key:
        return numbers_list[i]
else:
    return None

for i in range(len(numbers_list)):
    if numbers_list[i] > key:
        return numbers_list[i]
    else:
        return None

```

(b) HYPOCOMPASS offers three types of feedback to keep students engaged while improving their learning: (1) *Feedback to incorrect test cases*, ensuring students understand the code behavior; (2) *Feedback to correct explanations*, in the form of correct code fixes; (3) *Feedback to incorrect explanations*, in the form of confusion messages from the LLM-agent.

Section 5.2, we select the distractors to be close to, but distinguishable from, the correct explanation, such that they provide opportunities for students to clarify misconceptions.

To further improve students' learning, engagement, and help them correct misconception [49], HYPOCOMPASS also provides a set of **immediate feedback** (Figure 3b):

- **Feedback to incorrect test cases:** The accuracy of test cases is the cornerstone of the HYPOCOMPASS workflow and also serves as an indicator of users' comprehension of the problem. We take measures to prevent users from including incorrect test cases. For instance, when they submit pairs of inputs and outputs, we use a correct code solution to calculate the expected outputs and flag any discrepancy if they deviate from the user-written outputs.
- **Feedback to correct explanation selection:** When students select the right explanation for the bug, the LLM rewards the student by making the right edit. Importantly, as explained in Section 5.1, we carefully prevent over- or under- fixes to the

code, *i.e.*, we only make *local fixes* to the incorrect line(s) without touching other parts, such that the feedback can be meaningful to students and support interactive and gradual improvement.

- **Feedback to wrong explanation selection:** On the other hand, we also nudge students to improve when they select incorrect explanations: The LLM-agent will send messages expressing confusion and highlighting the discrepancy between explanations and the code behavior with a test case. The code behaviors are calculated based on the selected distractors (Section 5.2).

All the hints and feedback are visually displayed close to the next expected interaction of the student (*e.g.*, adding test cases, etc.) so as to minimize visual context switch. Whenever reasonable, we provide hints and feedback as the dialog messages in the OHQ interface in the tone of the LLM-agent, which reinforces the teachable agent context and helps students stay engaged in their TA role play.

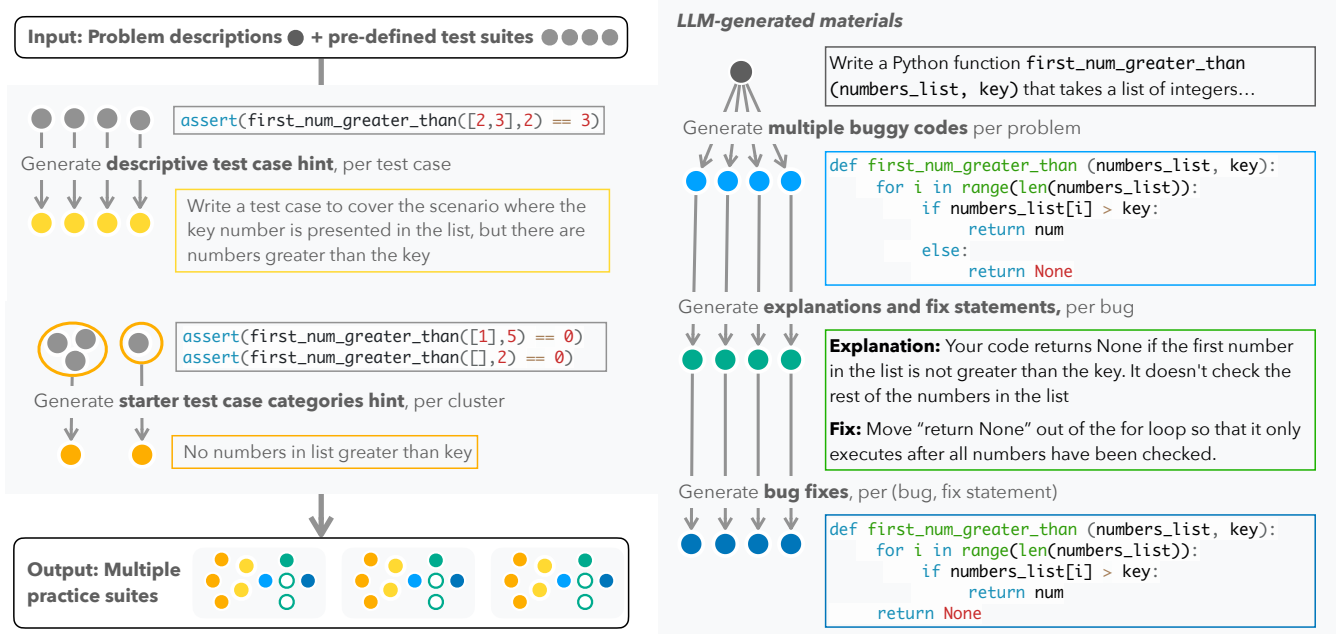


Figure 4: Examples of inputs and outputs to the LLM material generation pipeline. Given a program exercise description and its corresponding pre-defined test suite, HYPOCOMPASS generates all the required training materials in an end-to-end manner. The test case and test category hints are generated independently, whereas materials relevant to bugs are generated sequentially.

5 LLM INTEGRATION

As can be seen in Figure 2C₂, we use LLM to generate five types of materials: (1) starter test case category hints, (2) descriptive test case hints, (3) buggy codes, (4) explanations of bugs, and (5) codes with the bugs fixed.

We design the generation to be as lightweight for instructors as possible (R.4), in two ways. First, we **minimize the number of inputs required** and generate multiple *practice suites* (a package of the materials that can support a complete practice cycle) in an *end-to-end manner*. They are based on a textual description of a programming exercise, a corresponding reference solution, and a reference test suite with 5+ test inputs (which is readily available for most, if not all, graded programming assignments). Second, we **minimize the overhead of human verification**, by carefully designing the generation flow, optimizing the prompts, and utilizing automatic algorithms whenever possible.

Figure 4 provides more detailed steps of the end-to-end generation flow, with concrete examples per step. To balance the generation cost (in terms of generation time and expenses) and material quality, we use less advanced models in early prototypes and only switch to more advanced models when the performance from smaller models is consistently unsatisfactory. As a result, we rely on the OpenAI API for gpt-3.5-turbo for all materials except for explanation generation, which uses GPT-4 as the task requires the model to have better reasoning capability. We use prompts and temperatures in Table 1, and always take the top-1 output from the LLM. Note that all the prompts are run through the OpenAI chat API, as a dialog format can better preserve contexts when multiple

runs of LLMs are preferred.² Below, we present key factors to the success of the generation workflow.

5.1 Task formation and decomposition

Instead of naively describing the tasks in instructions, we carefully iterate on the prompts by utilizing different best practices. Three strategies stand out:

Avoid introducing competing tasks. The task that users specify through prompting can conflict with the inherent training objectives of LLMs [81]. In those cases, the external prompting context can compete with the LLM’s internal, parametric knowledge and, unfortunately, prior work has flagged that LLMs can be persistent with their internal beliefs at times [70, 90].

To stabilize LLM performance, we try our best to avoid introducing competing tasks unintentionally. Take the generation of **bug fixes** as an example. A vanilla approach should take the *buggy code* and the *fix statement* in Figure 4, and directly generate the complete *bug fix*. However, in experiments, we find that this vanilla strategy almost always leads the LLM to perform *over-fix*, i.e., generate codes with all the bugs fixed, regardless of what fix instruction is given. We suspect this is caused by an inherent competing task of *code completion*, i.e., as the LLM autoregressively generates tokens towards a complete code, the LLM is gradually biased towards (*correct*) *code completion* (which is part of the LLM pre-training objective) and away from *precise bug fixing* (the desired task).

To remove the code completion from the equation, we instead frame this generation as a *translation task*, translating bug-fixing

²We rely on OpenAI LLMs for their stability and generation efficiency, but plan to experiment with open models like LLaMa-2 [82].

Material	Generation goal	Model	T
Test case category hint [Sys.] You are a helpful and experienced teaching assistant of an introductory programming class. [User] Problem Description: {problem_description}. List three most important aspects of this problem that need to be tested by describing the type of input. Write only each aspect in 3-6 words [LLM] {A list of test category names}	To generate test categories that are meaningful for the programming exercise.	GPT-3.5	0.3
Test case desc. hint [Sys.] You are a helpful and experienced teaching assistant of an introductory programming class. [User] Problem Description: {problem_description}. Briefly describe this test case’s input and explain what important aspect of this problem that the following test case covers: {test_case} [LLM] {A description of the key aspect} [User] Reformat it as a one-sentence hint. Use this template: Write a test case to cover the scenario where ... [LLM] {One sentence hint in the right format}	To accurately describe the key aspect a test case is covering.	GPT-3.5	0.1
Buggy code [Sys.] You are a novice student in intro CS , you make mistakes and write buggy code when solving a problem. [User] ### Problem Description: {problem_description} ### Instruction: Write different buggy solutions with common mistakes like novice students. ### Buggy Implementations: [LLM] {buggy code #1} [LLM] {buggy code #2} [LLM] {...}	To over-generate bugs with mixed quality for selecting behaviorally distinct practice codes.	GPT-3.5	0.7
Bug explanation and fix instruction [Sys.] You are a helpful and experienced teaching assistant of an introductory programming class. [User] Hi, I’m a student in your class. I’m having trouble with this problem in the programming assignment: {problem_description} I’ve tried to fix my code but I’m still stuck. Can you help me? [LLM] Sure, let’s take a look at your code. [User] Here’s my buggy code: {buggy_code} What’s wrong with my code? List all the unique bugs included, but do not make up bugs. For each point, put in the format of: {explanation: accurate and concise explanation of what the code does and what the bug is, for a novice, fix: how to fix the bug, within 30 words} Only return the bullet list . Do not write any other text or code. [LLM] {Bullet list of json formats with explanation and fixes}	To describe each unique bug, and write a corresponding fix instruction. If there are multiple bugs in the code, generate their explanations and fixes separately.	GPT-4	0.3
Bug fix [Sys.] You fix bugs in Python code closely following the instructions. [User] Original code: {buggy_code}; Code modification: {explanation} Translate the statement into actual, minimal code change in this format: {original code snippet: ""copy the lines of code that need editing"" -> edited code snippet: ""write the edited code snippet""} [LLM] {JSON formatted old to new snippet, e.g., numbers_list[i] <= key → numbers_list[i] > key } [Sys.] You fix bugs in Python code closely following the instructions. [User] Old Code:{buggy_code}; Instruction:{Old snippet to new snippet}; New Code: [LLM] {The complete version of the new fixed code}	To edit the buggy code according to the fix instruction, without over- or under- fix.	GPT-3.5	0.3

Table 1: Parameters used for generating training materials: prompts, the LLM model used, and the corresponding temperature.

instructions to its code format of old code snippet → new code snippet (Table 1). With the LLM generating only part of the codes in a strict format, it is “forced” more toward the user task. Empirically, we find that the translation approach greatly improves *the locality of bug fixes*, and reduces the over-fixing error from about 100% to 30% when there are multiple bugs in code (evaluation details in Section 6).

Provide explicit prioritization of requirements. Excessive requirements in one LLM prompt can cause the model to selectively perform a subset of them [88]. For example, the prompt for generating **bug explanations and fix instructions** requires the LLM to (1) find all bugs, (2) write explanations and fixes to be accurate and concise (to be readable and meaningful even when presented with

distractor explanations), (3) and avoid hallucination. Describing all these criteria in the same paragraph produces a success rate of around 50%, even with GPT-4. Instead, we find that it can be effective to *explicitly phrasing certain criteria to be more primary*. To primarily enforce the precise bug extraction, in the prompt in Table 1, we instruct GPT-4 to *list all the unique bugs included* and *not make up bugs* upfront. Then, we specify secondary requirements only inside the *output format* (e.g., *concise explanation, within 30 words*). Empirically, we find this hierarchical disentanglement helps the model better capture and describe all the unique bugs without hallucination, increasing the success rate to >90%.

Build LLM-chains for multi-step tasks. Several tasks can easily benefit from a LLM-chain-based design [87], *i.e.*, to decompose the overarching task into multiple sub-tasks, each assigned to a separate LLM module, then combined. Depending on the task objective, there are two types of chains to employ: (1) *Multi-step reasoning*: We decompose the generation tasks that have clear sub-steps for more stable performance. For example, we generate **test case hints** by first describing the key aspect a test case covers, and then rewriting it into a one-sentence hint format. (2) *Iterative refinement*: Taking advantage of the models’ sensitivity to prompts that involve comparisons [88], we ask models to generate responses iteratively instead of all in one run, each time asking for “different” things (*e.g.*, we ask for “different” common mistakes in buggy code generation for multiple rounds). This is akin to the idea of Self-Refine [61], using LLMs to improve on their own qualities.

5.2 Over-generate-then-select

While LLMs can easily generate random materials, it is nontrivial to ensure these generations have pedagogical values. For example, to increase the utility of the practice, we would like to expose students to *behaviorally distinct* bugs, a property that is hard to enforce through pure prompting as it requires the LLM to “know” the behavior of each buggy code. Instead, we exploit the non-deterministic nature of LLMs, configure them to over-generate multiple solutions with mixed qualities [60], and then select a subset of desired ones. As in Figure 5, we use this strategy in three parts.

First, to **expose students to behaviorally distinct bugs**, we over-generate buggy codes per programming exercise, then select n of them as the *diverse practice code* (Figure 5B). To do so, we vectorize the behavior of each buggy code using its behaviors on the reference test suite, with each number being 0 (passed the test) or 1 (failed the test). We filter out the correct codes (defined as all test cases passed), and then greedily select $n = 3$ practice buggy codes that have the maximum distance against each other, using Euclidean distance on codes’ behavioral error vectors.

Second, to select candidate bug explanations that **help students clarify misconceptions** (Figure 5C), we select $m = 2$ distracting explanations for each practice buggy code, which should look similar to the actual explanation. To do so, we again rely on the over-generated buggy codes and their behavioral error vectors. We select the buggy codes whose error vectors have the smallest non-zero Euclidean distance with the targeting practice code, and use their corresponding explanations as the distractors. The mapping between distractor codes and explanations also helps us generate the confusion messages (Figure 3b) — Whenever a student selects the wrong explanation, we use its corresponding distractor buggy code to find test cases that can be surfaced to students.³

Third, to ensure our test case category hints **capture key testing aspects that matter for the buggy code distribution** (Figure 5D), when there are enough reference test cases (*e.g.*, 10+), we cluster them into semantically meaningful groups, and use them to guide

³Note that besides error vector distance, we also experimented using the code syntax distances calculated with AST tree distance and RougeLCSUM (as motivated by [50]). While these methods prioritize codes with syntax differences, we find that it is possible to have codes that look different but behave the same. We decided to stick to the error vector distance as it allows us to focus on high-level code behaviors instead of implementation details.

the selection and editing on the test category hints. Specifically, similar to our aforementioned representation of the code, we vectorize each test case by the behaviors of different codes on it, which can be visualized as the columns in Figure 5A. We run Hierarchical Agglomerative Clustering (HAC) [56] on these test case vectors, and select the clusters based on the dendrogram visualization.

5.3 Human-in-the-loop verification

As in Figure 4, while the hints for test cases and categories are generated separately, the materials relevant to bugs are generated in sequential order. We perform human verification per step, which not only ensures immediate high-quality generation, but also mitigates the risk of cascading errors in subsequent steps. For buggy code, human editing means removing extra comment lines from the generated code; while for others, it means revising incorrect snippets in the generated material or ensuring the material follows certain formats that enable subsequent parsing. We provide more details on human verification and human editing times in Section 6. As of now, all the materials in HYPOCOMPASS are *pre-generated* in order to minimize the risk of students being misled by incorrect LLM responses.

6 INTRINSIC EVALUATION: LLM-GENERATED MATERIAL QUALITY

To assess the quality of LLM generations and whether they support our training loop, we evaluate different types of LLM generations with expert labels and automatic measures.

Material generation. In total, we evaluated the generation on six different practice problems (detailed descriptions in Appendix A), namely (1) `first_num_greater_than` (the example in Figure 1), (2) `remove_extras`: remove repeated occurrences of elements in lists; (3) `num_smaller`: count the elements in a list that is smaller than a given number; (4) `sort_age`: sort people by their age; (5) `top_k`: find the greatest k numbers in a list; (6) `swap_keys_values`: swap keys and values in a dictionary.

These six problems are adapted from prior work [20, 42, 50] and our own CS1 problems, and each of them has a problem description, a reference solution, and a set of reference inputs (10 on average). We selected the problems that are suitable in a CS1 context, whose solutions can have diverse implementations but unique input-output pairs.

On average, for each problem, we generated 3 test category hints, 10 test case hints, 24 buggy codes (around 31% are codes with multiple bugs), and, accordingly, 24 explanation and fix instructions, and 33 bug fixes. The total number of materials used in the evaluation is shown in Table 2.

Evaluation metrics. We summarize success rates across all problems in Table 2, and we provide the definitions of success criteria for different material generation steps in Table 4 in Appendix A.

- *Test case category hint*: A good test case category should be *reasonable to test for a problem*. We define the success rate as the proportion of *suitable test categories* over all the test categories per problem.

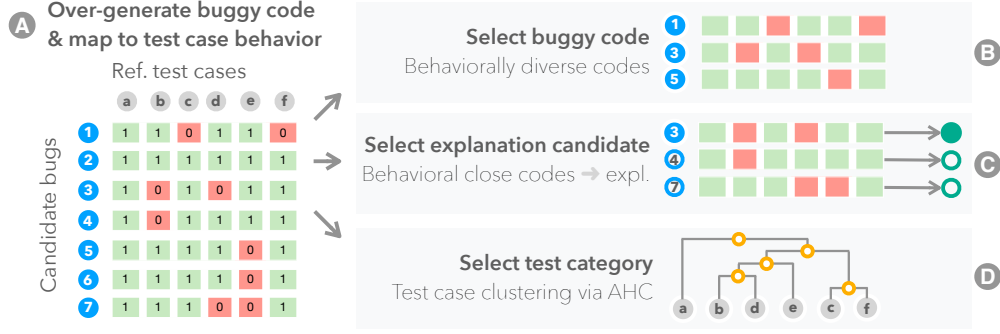


Figure 5: We over-generate candidate materials and automatically select those that have pedagogical values. Specifically, we (A) over-generate potential buggy codes to a programming exercise and calculate behavioral vectors using these buggy codes and reference test cases. Then, we (B) select buggy codes with distinct behaviors on tests for practice, (C) select distractor codes that behave similarly for the explanation pool, and (D) cluster reference test cases to guide the revision of test category hints.

Material	Raw LLM outputs			Human verification		
	# Generation	Avg. gen time	Success%	Avg. edit time	IRR%	κ
Test case description hint	61	0:00:37	98.36%	0:00:08	100%	-
Test case category hint	18	0:00:10	94.44%	0:00:10	100%	-
Buggy code	145	0:01:30	57.93%	0:00:02	n/a	-
Bug explanation and fix instruction	145	0:03:36	91.72%	0:00:52	83%	0.783
Bug fix	195	0:02:45	86.15%	0:00:37	96%	0.780

Table 2: LLM Evaluation: Time, Success rate, and Inter-Rater Reliability scores (i.e., $IRR\% = \#agreements / \#total\ labels$, κ means Cohen’s Kappa coefficient). LLM can consistently generate high-quality requirements that can be manually verified with moderate agreement and edited in a short amount of time.

- *Test case description hint*: A good test case hint should accurately describe the test case behavior. We define the success rate as the proportion of test case hints that provide an accurate description.
- *Buggy codes*: Intuitively, we prefer buggy codes that add more meaningful variations to existing buggy code collections. Therefore, we define success rate as the ratio of behaviorally distinct codes, where code behavior is automatically categorized with instructor-provided reference test inputs.
- *Bug explanation and fix instruction*: As the prompt requires identifying multiple bugs, we manually evaluate each pair of explanation and fix instructions, and count one generation as correct when there is a matching explanation and the corresponding fix, and both are correct.⁴
- *Bug fix*: We define a bug fix to be a success if the { old code snippet → new code snippet } accurately reflects the instruction without over- or under- fix.

Human labeling procedure. We perform manual inspections for most materials except for judging whether a code is buggy, which can be evaluated by automatically running the generated code against the reference test suite.

Two authors annotated 10% of the generations at each step individually and calculated the inter-rater reliability (IRR), which is satisfactory across different model generations, as summarized in the IRR% and κ columns of Table 2. Note that for buggy code, there is no need for manual labeling, so the entry is n/a. Also, when both

⁴In some cases the LLM describes the same bug twice because there are conceptual differences between them even when they actually share the same fix. We still count this as correct, as they only require humans to delete the extra bugs.

raters agree 100% in one category, Cohen’s kappa is undefined, so κ is noted only with the percent IRR is not 100%.

One author annotated the rest of the generated materials, and the success rate is calculated based on this author’s annotation. Note that if generation takes in the prior model’s outputs as inputs, for example, generating fixed codes takes in the previously generated bug explanation as input, the manual evaluation and editing step will be done before the input feeds into the model to ensure that the model input is always correct. We further log the verification and editing time (summarized in Table 2), as proxies to the instructor overhead. More details on human labels and edits are in Appendix A.

Result. The statistics of the number of generations, success rate, and average editing time are summarized in Table 2. Overall, we achieve high-quality generation: a complete set of practice materials with 9 buggy codes (3 for practice and 6 more as distractors), 9 bug explanations, 9 bug fixes, 10 test case hints, and 3 test category hints can be generated with a 90% success rate and only take 15 minutes to label and edit, which could significantly reduce instructor efforts compared to *status-quo* (Section 2).

More specifically, test-relevant hints generally have high quality and require minimal editing efforts from humans, mostly to make them more concise when needed. At first glance, buggy codes seem to have a low success rate of 58%. However, note that as we over-generate buggy codes before selection, empirically having a number over 50% is more than reasonable for practical use.

Codes with multiple bugs tend to be the biggest cause of inaccuracy in generating bug explanations and fixes. LLMs tend to hallucinate bugs, incorrectly map explanations and fix instructions

to bugs, and over-fix all the bugs in a code when the instruction only specifies one bug to fix. Our prompting strategies in Section 5.1 greatly improve the material quality. For example, with bug fixing first framed as a translation task, we are able to reduce the error to 29.79% in multi-bug scenarios (near 100% without this framing) and 9.66% out of all buggy codes. This is still the biggest cause of inaccuracy in code fixes, but fortunately, these issues are not hard to manually fix, as most of the time the correct answer is already in the generation, so we simply need to remove the incorrect parts. On average, labeling each buggy code’s explanation and fix instruction and the bug fix takes 1.5 minutes.

We notice some disagreements in annotation, especially around the definition of multi-bugs in the context of HYPOCOMPASS (e.g., two conceptually different bugs can be fixed simultaneously which defeats the purpose of gradual code-fixing interaction). We resolve to use single-bug codes in our user study and will investigate more sophisticated designs around multiple bugs in the future.

7 LEARNING EVALUATION: USER STUDY WITH PRE- / POST-TEST

Can novices formulate hypotheses more comprehensively and accurately after engaging with HYPOCOMPASS? We conducted a user study with 12 students and compared the difference in speed and performance from the pre-test to the post-test on an assessment designed in accordance with our learning objectives.

7.1 Study Design

Evaluation metric. We assessed student learning gains using a pre-to-post test, a common evaluation method for instructional designs [78]. Students take a *pre-test* before receiving instructions and then a *post-test* afterward. Both tests should align with the learning objectives and maintain a similar level of difficulty.

Existing methods for assessing students’ debugging skills often prioritize the speed and accuracy of bug fixing, rather than focusing on hypothesizing and identifying bugs (LO2). While testing against all students’ buggy code seems to be a satisfactory measure of the comprehensiveness (LO1) of their test suites [25, 26], this approach is not universally applicable and fails to reveal whether students generate test cases randomly or systematically.

To best capture student learning gains on our learning objectives, we took a backward design method [86] to create an aligned assessment for the comprehensive LO1 and accurate LO2 hypothesis construction skills in debugging.

We conducted multiple rounds of pilots (P1-8, $n = 8$) to refine our HYPOCOMPASS intervention and pre-post tests. In these iterations, we created both a multiple-choice and an open-ended version of the assessment. In the pilot tests, we observed that both question formats exhibited similar completion times and accuracy rates. As such, we used the multiple-choice version to simplify evaluation.

Our final tests are based on two programming exercises with similar levels of difficulty: `remove_extras` and `num_smaller` (also used in Section 6). Each test consists of seven questions, with three items related to LO1 and four related to LO2. Figure 6 provides a sample for each question. For instance, Question 3.1 presents students with a test suite and asks them to identify a more suitable

test case to include, thereby assessing their ability to construct comprehensive hypotheses (LO1).

We measure student performances using their **test scores**, which an author manually graded based on a standard rubric. We also log the **completion time** as a proxy for their proficiency.

Participants. We recruited undergraduate and graduate students with various backgrounds from a private research institution in the US. We aimed to recruit novice students who have basic knowledge of Python (the language used in our experiment) but are not proficient. To achieve this, we advertised our study on mailing lists with appropriate student distributions and within four relevant courses. Students interested in the study were first asked to complete a screening survey, in which we gathered information about their prior programming experiences and requested that to complete the programming exercises `first_num_greater_than`. This problem served as one of the exercises in our study. We excluded students who have extensive programming experience (e.g., have completed courses like Computer Architecture) or correctly solved the coding exercise in less than 3 minutes (who were typically advanced developers during our pilot studies). Out of the 28 students who completed the survey, we conducted studies with 12 that met our requirements (U1-12) — 9 females, 3 males, and 7 non-native English speakers with an average age of 22.5.

Study Procedure. Our hour-long user study constituted a pre-survey, pre-test, interaction with HYPOCOMPASS, post-test, and a post-survey. A participant was first asked to complete a *pre-survey*, with demographic information and 7-level Likert Scale questions on their past experiences on debugging and writing test cases. Then, they were given up to 20 minutes to complete the pre-test, which is empirically enough for most students (average completion time $15 : 22 \pm 3 : 22$). Upon completion, the participant interacted with HYPOCOMPASS for about 25 minutes, before spending another up to 20 minutes on the post-test.

The system interaction consisted of 2 problems, where for each of the problems, the participant needed to write a test suite and explain the bugs in 3 different buggy codes (as elaborated in Section 4). The first problem was the same as the pre-test problem, and the second problem was the same as what the students saw in the screening survey (`first_num_greater_than`), which helped better isolate student learning towards our learning objectives from the program comprehension skills.

Participants completed the study by filling in another post-survey with Likert Scale and open-ended questions on their experience using HYPOCOMPASS. We asked whether their test cases correspond to certain hypotheses they have in mind, whether they found the code fixes reasonable, whether they think the buggy code are written by AI or student, how engaging, fun, and frustrating was the interaction, and whether they found HYPOCOMPASS helpful in learning about debugging. Upon completion, the participants were rewarded with a \$15 Amazon Gift Card for their time.

We counterbalanced the problem used for pre-post tests to control for the influence of problem sequence; we found that two problems have comparable difficulties from pre-test results (an average of 65% correctness on `remove_extras` and 61% on `num_smaller`).

Question 2 Select all that apply.

Imagine a solution to this problem that fails on all of these following test case(s) because of the same bug. What could the bug be?

```
Python
assert(num_smaller([10, 10, 10, 20, 30], 10) == 0)
assert(num_smaller([10, 20, 20, 30, 30], 20) == 1)
assert(num_smaller([10, 10, 20, 30, 30], 30) == 3)
```

Select all that apply.

- A. The buggy codes may have overlooked duplicated elements in seq
- B. The buggy codes may have overlooked x when it is the smallest element in seq
- C. The buggy codes may only handle the case where x is not in seq
- D. The buggy codes may only handle the case where x is already in seq

Answer:

Question 3.1 Select one answer.

Given this current test suite, which test case out of the pair, if added to the test suite, can cover additional aspects of the problem and make the testing more comprehensive? In other words, which test case is more likely to reveal an issue or bug in a code solution that this current test suite would not reveal?

```
Python
assert(num_smaller([10, 10, 10, 20, 30], 10) == 0)
assert(num_smaller([10, 20, 20, 30, 30], 20) == 1)
assert(num_smaller([10, 10, 20, 30, 30], 30) == 3)
```

A. `assert(num_smaller([1, 3, 5, 7], 5) == 2)`
 B. `assert(num_smaller([10, 10, 20, 30, 30], 40) == 5)`

Question 7 Select one answer.

Test case 2: `assert(remove_extras_code2([1, 1, 2, 3]) == [1, 2, 3])`
 Actual behavior: `'TypeError' 'int' object is not iterable.`

```
Python
1 def remove_extras_code2(lst):
2     new_lst = []
3     for i in lst:
4         if i == lst[i+1]:
5             continue
6         else:
7             new_lst += i
8     return new_lst
```

What's the bug exposed by this test case?

- A. The bug occurs because the loop variable `i` is mistakenly used as both the element and index of the list. This leads to incorrect comparisons and triggers a `TypeError` in `lst[i+1]` because `i` is an element of the list, not an index.
- B. The bug is caused by not initializing the `new_lst` properly. The code fails to explicitly assign an empty list to `new_lst`, so when concatenating elements to `new_lst` using the `+=` operator, a `TypeError` occurs because `new_lst` is not iterable.
- C. The bug is due to an incorrect conditional statement. The code incorrectly compares `i` with `lst[i+1]` instead of comparing adjacent elements of the list, which triggers `TypeError` when trying to compare an integer `i` with a list element.
- D. The bug occurs because the code incorrectly assumes that `i` is iterable when concatenating it to `new_lst` with the `+=` operator. In this case, `i` is an integer, which is not iterable, and it causes a `TypeError`.

Answer:

Figure 6: Pre-post test question examples. Q2 and Q3.1 correspond to comprehensive hypothesis construction LO1, and Q7 correspond to accurate hypothesis construction LO2.

7.2 Quantitative Learning Gains: HYPOCOMPASS Supports Learning

For the quantitative analysis, we removed two students from the analysis whose pre-test scores were above 95% (6.75/7), indicating there was not much room for improvement for them in our study.

HYPOCOMPASS can help students improve on hypothesis construction. Using a two-tailed paired t-test, we found that the pre-test to post-test performance of all 10 participants with (pre-test scores lower than 95%) significantly improved by 16.79% ($p = 0.044 < 0.05$), and the time of completion significantly reduced by 12.82% ($p = 0.004$), indicating success in learning through HYPOCOMPASS interaction.

Students showed improvement in both learning objectives, with a more significant enhancement in accurate hypothesis construction. Where does the observed improvement come from? We further break down the analyses by learning objectives. We found a marginal 2.50% improvement in the score ($p = 0.755$) and a significant 26.05% time reduction ($p = 0.004$) for *comprehensive hypothesis construction* (LO1), and a significant 27.50% improvement in the score ($p = 0.032$) and a marginal 7.09% time reduction ($p = 0.262$) for *accurate hypothesis construction* (LO2).

Our result suggested that after interacting with HYPOCOMPASS, novice students could construct comprehensive hypotheses about what aspects to test in code more efficiently. They also showed an improved ability to construct hypotheses about what might be causing a bug in the code when faced with a failed test. Note that these significance values confound with problem difficulty: it appeared that the items corresponding to LO1 (pre-test mean = 68%) were easier than the ones for LO2 (pre-test mean = 34%).

Preliminary observations showed that HYPOCOMPASS may be more beneficial than *status-quo*, but more rigorous testing is required. We primarily designed this study to understand whether the setup of HYPOCOMPASS provides any learning gains. As a result, we focused our (admittedly challenging) recruitment effort on having students interact with HYPOCOMPASS. That said, we did conduct four pilot studies on two different control conditions ($n = 2$ each), mostly for evaluating our pre-/post-test design. One control involved using the same sequence of practice materials generated by LLMs without HYPOCOMPASS (control-llm), and the other one involved the conventional debugging practice adopted from a CS1 class materials (control-conventional). We found preliminary evidence that students' pre-post performance and time improved more in control-llm than control-conventional, but both not as much as the significant improvement of the system condition. We plan to conduct a more rigorous controlled comparison with more participants when the semester progresses to a point where CS1 students are ready to receive debugging training.

7.3 Student Perceptions: Effective and More Engaging Learning

Through analyses of the survey responses and follow-up comments, we further unpack the reasons behind quantitative improvements. For the statistical analysis of Likert Scale questions below, we use the Wilcoxon signed-rank test for the ratings that come from the same participants as the non-parametric version of the paired t-test, and we use the Mann-Whitney U test when the ratings come from different participants. Note that for the sake of discussing alternative designs, we also report reflections from several pilot users, but their survey responses are not included in the analysis.

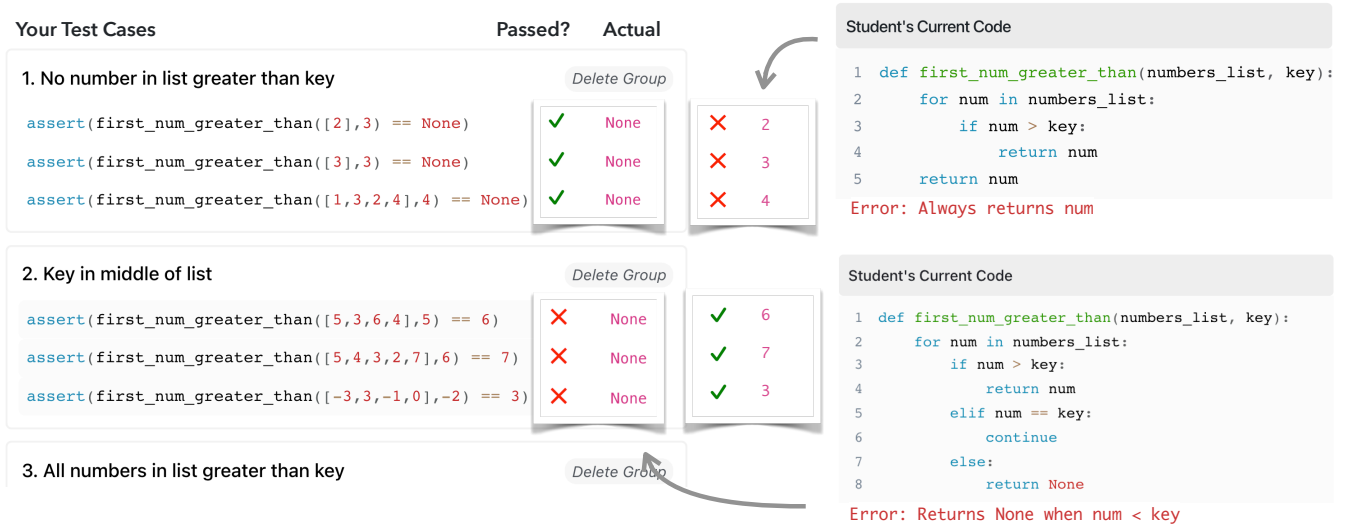


Figure 7: U3's test suite which demonstrated different pass/fail patterns corresponding to different bugs.

Students self-perceived to learn more about hypothesis-driven testing and become more motivated to write test cases systematically. When asked “What do you think you’ve learned (if any) through this interaction,” eight participants stated that they learned about the importance of hypothesizing with test cases. For example, U2 commented that “I learned about the importance of creating good test cases that have specific hypotheses. They really help to streamline the process of debugging”; and P3 said “I’ve implemented different cases relating to problems in ways that I might otherwise not have come up with.”

The test case categorization hint and interaction were also useful. It not only made comprehensive hypothesis construction (LO1) easier, but also supported accurate hypothesis construction (LO2) – With the explicit nudge of categorization, different bug patterns became more explicit. For example, P3’s test suite displayed different pass/fail patterns with tests nicely organized into categories, intuitively suggesting what the bug might be for different bugs (Figure 7). P3 also commented that “categorizing the test cases was very helpful in finding the type of bug.”

Moreover, HYPOCOMPASS motivated students to write more test cases: participants strongly agreed with the statement “I want to write more test cases when I’m solving future problems” (with an average score of 6.25 ± 0.87 out of 7). Participants commented that they saw more value in having a comprehensive test suite because HYPOCOMPASS make them “classify test cases with specific names” (U11), “see bugs slip away from my tests” (U4) and “[observe] different test cases fail different codes” (U9), which make test cases more helpful and relevant to their debugging process.

Students felt well-scaffolded in HYPOCOMPASS with explanation options, correctness feedback, and hints, but some prefer less help. Two participants found it was very helpful to have explanation options provided. For example, U1 said “I feel such a relief that I don’t need to debug myself, even when I look at the code

and think it looks right, looking at the test behavior and the explanation options really helps relieve that burden.” One participant also found the hint and correctness feedback to be useful, as “otherwise [there is] no safeguard if you enter wrong test case or explanation to choose so easier to make mistakes.” (U4).

Our pilot study users also appreciated the explanation, hint, and automatic fix. For example, P6 commented, “When I see the feedback when my explanation is incorrect and re-select the right one and see the fix corresponding to that, it really helps me understand what’s wrong in the code. Hint is also good since it reminds me to self-reflect.” P4 added, “I really like it since I can just leave the Python debug part to it, so I can just find the problem and evaluate the fix automatically instead of climbing on a whiteboard to wonder and process all the info. ... it’s more convenient than changing it manually.”

We also received valuable suggestions including improving UI design for hints and instructions location (3), and enabling more open-ended interactions without scaffold (4), such as letting users enter their own explanations and hiding the test case categories hints. In three of our pilots, we tested fading scaffolding by initially offering both the explanation pool and the option to type their own explanation and eventually requiring users to enter their own explanations. The pilot users had quite different reactions: P7 typed their own explanations from the start, and P6 used the explanation pool when available and adapted quickly when required to type their own explanations. However, P8 struggled with generating their own explanation and became frustrated when the model could not provide the correct fix. To support students with varying needs, we can offer different support levels as options but should always have a fallback solution. We discuss more alternatives in Section 9.

The LLM-generated bugs and fixes felt authentic to students. As commented by F4 in our formative study, it is important to create “authentic debugging exercises that feel genuine” to make debugging relevant and motivating to the students. In HYPOCOMPASS, LLM-generated bugs and bug fixes feel natural and authentic

to students. For example, students agreed with the statement that “the fixed code reasonably reflects the explanation” (5.58 ± 0.79), reflecting that using LLMs to offload code completion skills is a promising direction for enabling interactive debugging.

When asked “Do you think the buggy solutions you’ve seen are your fellow students’ submissions or AI-generated? Why?” half of the participants answered Cannot tell. They attributed their selection to experiences making or seeing similar mistakes from peers (3) and not enough programming experiences (2). For example, U11 mentioned “I wrote some code that was similar to these, and I saw some peers making these mistakes”. The other half mostly selected AI’s code due to their weird logic (2), clean writing style (2), ease of fix (1), and “because you asked this question” (U1). These observations echo prior observations that LLMs can exhibit common mistakes similar to humans [29], and that it is possible to use LLM-generated code to automatically create practice that feels authentic. Interestingly, participants have different expectations of AI’s code. For example, U6 believes that the readability of code is a sign that this can be written by a human student, while U10 believes that it is AI’s code because “the indentations are always correct and the code is neat”.

HYPOCOMPASS supports students’ engagement and confidence in debugging. We saw that students’ confidence with regard to debugging increased significantly after interacting with HYPOCOMPASS. For the question “How confident are you in terms of your debugging ability?” that occurred both on pre-survey and post-survey, a Wilcoxon signed-rank test shows a significant increase by 13% among all 12 participants ($p = 0.016$).

Additionally, we asked all users to rate the engagement, fun, and frustration aspects of HYPOCOMPASS on a scale of 1 to 7; we also showed them a brief demo of a control intervention representing conventional debugging practice with static handouts and asked for their ratings on the same three dimensions. Participants rated HYPOCOMPASS as significantly more engaging (6.25 out of 7), fun (6.25), and less frustrating (1.75) than the control intervention (4.75, 4.75, and 2.5 respectively, with $p < 0.005$ for each). Note that we also asked the aforementioned control pilots to rate the engagement, fun, and frustration aspects of HYPOCOMPASS. In general, they rate the system as better than the intervention they received as well, but to a lesser extent that does not reach statistical significance, so there could be some exposure bias in the system users.

During the debrief, participants commented that HYPOCOMPASS is “actually surprisingly fun” (U1), “super cool” (U7), and “want to see it in the class” (U3). U8 especially liked the role-play framework: “the role play just feel more natural because it feels like explaining to a rubber duck instead of talking to myself.” U3 found the tool friendly and helpful: “It’s really an amazing tool! The design is super friendly to anyone, and I like that there are some categories and we can organize test cases. I have always been writing random test cases and hoping they catch bugs, which is not very efficient. But this tool is very helpful. And the OHQ is cute!”

7.4 Feedback from Instructors and Education Researchers

We also did pilots and/or interviews with three CS1 course TAs, three instructors, and two education researchers at our institution

(T1-8) and surveyed three instructors in our formative study to ask about their thoughts on the potential use of HYPOCOMPASS in classes. The feedback is positive in general, and instructors also see more practical considerations of where it fits into the current class and the potential use of HYPOCOMPASS for TA training.

Practical Considerations for Integrating into Class. Instructors who saw the system also found it potentially engaging for students, and many expressed willingness to try HYPOCOMPASS in class in different capacities, e.g., as extra practice or reflections on student quizzes. That said, instructors tend to make more feature requests for integrating HYPOCOMPASS into current instructions. For instance, T4 had a setup allowing students to rectify their incorrect quiz code submissions, and so wanted to explore if students could effectively utilize HYPOCOMPASS interface on their own code. Another instructor F4 had previously tried manually creating and labeled test suites for their students, finding the process time-consuming. As a result, they wished that HYPOCOMPASS could assist in automatically explaining the purpose of test cases. Meanwhile, a TA (T2) who liked to demonstrate print statements during office hours expressed, “I believe this is an effective way to encourage students to delve into writing test cases, which is essential in more advanced courses. However, I also want a feature with print statements that helps pinpoint exactly where an issue lies.”

On one hand, we could borrow insights from Guo [37]’s Python Tutor (one of the most widely used research software) if we want to sustain a large user base and share HYPOCOMPASS widely. On the other hand, easy adoption of a tool in class needs a lower barrier, and some instructors found partial usefulness in the materials generated by HYPOCOMPASS. As nicely put by F6: “To be honest, adding another tool to my system is expensive to me. It’s another thing students might get confused by, I might have to maintain somewhat, etc.” For them, directly providing LLM-generated practice suites as handouts seems more practical: “If you could take your thing and make it easy to adopt, like literally have it be things that could be printed out or equivalent.” We discuss modularizing HYPOCOMPASS in Section 9.

Potential use for TA Training. Since HYPOCOMPASS required users to role-play as TAs, instructors naturally saw the potential for using HYPOCOMPASS in TA training. Two instructors commented that it helped “make new TAs more comfortable before they talked to real students” and encouraged the TAs to contemplate how to make “incremental changes based on students’ solutions” rather than instructing them to rewrite the entire code. As HYPOCOMPASS naturally exposed users to alternative solutions and diverse bugs, it could be beneficial for both students and TAs [77].

Indeed, we observed some naturally occurring attempts to enhance teaching from the more experienced pilot users. For example, T7 experimented with different styles of explanation to make it more understandable and provided high-level explanations e.g., without the specific fix instruction (which, the model was able to follow). T3 began to pay more attention to test case selection, using the simplest possible inputs to illustrate the bugs, and also revised the wording multiple times to prevent confusion. However, it is important to note that HYPOCOMPASS was primarily designed for novice students to learn hypothesis construction skills and not as a TA development tool, such as GPTeach [62], which would require a different set of design considerations.

8 RELATED WORKS

8.1 Existing Instructions for Debugging and Hypothesis Construction

In the design of explicit instruction on bug finding, Katz and Anderson [46] analyzed bug finding instruction, identifying two strategies: program order (reasoning by execution order) and backward order (reasoning from output to bug origin). They examined the trade-off between speed and accuracy but found no performance improvement. In a similar approach, Whalley et al. [84] guided students with forward reasoning in hypothesis-driven debugging, involving steps like generalizing problems from failing tests, generating hypotheses, locating bugs, and proposing fixes. They observed difficulties in steps 2-4, with success in earlier steps correlated to later success in one of two problems. Both studies focused on the strategies and processes that may lead to more accurate hypotheses about bugs, but neither explored the comprehensive hypotheses construction aspect. In the software engineering education literature, Ahrendt et al. [3] covers testing, debugging, formal specification, test case generation, and verification in their class. But currently, there is still a lack of explicit, adaptable, and engaging instruction and assessment for hypothesis construction.

8.2 Existing Tutors and Tools for Debugging and Hypothesis Construction

Researchers have developed debugging tools and tutoring systems for deliberate practice and feedback. Examples include Ko and Myers [48], generating “why did” and “why didn’t” questions and answers. Carter [16] focuses on teaching Case-Based Reasoning for debugging. Luxton-Reilly et al. [57] guides bug fixing systematically with forward and backward stepping. Lee et al. [53]’s ViDA offers test case annotations for bug hypothesizing. Ardimento et al. [7]’s DTE in Eclipse provides visualization scaffolds for bug causality. Ecanow [24]’s Debug Tutor offers VS-Code debugger exercises. None of these tools targeted or assessed the learning of comprehensive hypothesis construction skills. They often require manual effort and are language-restricted, making dynamic adaptation challenging. Engagement and preference evidence were lacking for many systems too [7, 16, 24, 53].

8.3 Large-Language Model Capabilities

Previous research explored various capabilities of LLMs, such as auto-completing code alternatives [93, 98], generating documentation and synthesis [9, 96], writing test cases [52], auto-creating and fixing bugs [5, 38, 74], providing algebraic or bug fix hints [72, 74], offering feedback based on rubrics [19], simulating students [62], among others. GPT-3-based models perform averagely in CS1 tasks, with limitations in complex code [74, 80, 93]. GPT-4 performs better in CS1 problems but is not on par with practicing engineers [14]. Despite impressive capabilities, LLMs still produce bugs, including syntax errors and wrong function calls [30]. These bugs can be used to improve model performance [45, 54], but there is no study utilizing LLM-generated bugs as educational opportunities. Additionally, to enhance stable and good quality generation of bug explanations, techniques like chaining output as next input [87] and various prompting methods [55] can be employed.

9 DISCUSSION AND CONCLUSION

In this work, we build a theoretically motivated and empirically tested debugging tutor augmented by LLM, which aims to provide engaging and deliberate practice on novices’ comprehensive and accurate hypothesis construction skills. Our evaluations show that HYPOCOMPASS can stably generate high-quality materials, and can effectively train novices on comprehensive and accurate hypothesis construction. Still, HYPOCOMPASS has room for improvement in personalizability and interactivity. Below, we reflect back to our design rationales in Section 3.2, and discuss our main takeaways from this study, observed limitations, and corresponding next steps.

9.1 Safe and Appropriate LLM Exposure (R.1): Train Humans for Appropriate Reliance, with Imperfect AIs

While LLMs and other AI systems have faced criticism for their brittleness [35, 68], our HYPOCOMPASS illustrates a scenario in which *LLM mistakes are not seen as bugs, but rather as features*. This setup is important as it facilitates the practical utility of inevitably imperfect AI models, allowing both students and instructors to reap the benefits of LLM advancements without hindering student learning. Furthermore, it has the potential to provide students with controlled and moderated exposure to imperfect LLMs. This exposure enables them to construct a reasonable mental model of LLM mistakes before encountering them in uncontrolled, real-world scenarios.

The current version of HYPOCOMPASS remains quite conservative, in the sense that it explicitly utilizes LLM-generated bugs for training, but all materials undergo initial offline generation and validation before being released to the students. In other words, we mitigate the risk of overtrusting LLMs by *preventing the model from making mistakes*. As a result, we have achieved *practical usability* but not *adequate exposure*. We make this design choice because it is in fact quite challenging to strike a balance between providing reliable instruction and exposing students to LLMs. This challenge is especially pronounced because our students are novices and are more susceptible to LLM-induced bias — We are concerned that discrepancies between student actions and LLM feedback may reinforce students’ incorrect beliefs. Moreover, nonsensical LLM responses could potentially frustrate or demotivate the students.

That said, our intrinsic evaluation in Section 6 shows that the material generation has a reasonable success rate and could potentially support real-time interaction to fully tap into the potential of HYPOCOMPASS’s teachable agent setup. We plan to explore more direct interactions between students and LLMs, e.g., having students propose their own explanations for bug fixing, which is a also popular request from our study participants. The setup will require a more sophisticated design around scaffolding students to recognize LLM mistakes and address the aforementioned risks, as our evaluation shows that when there are multiple bugs in code, the model can still over-fix 10% of the time even given an accurate explanation and fix instruction. For example, along with the bug fix, we can use the LLM to provide a description of how the code-change impacts the program behavior, and use those to help students realize discrepancies between the actual bug fix and their self-proposed instructions. We can also let the student attempt to explain bugs themselves first as an active learning opportunity,

but provide the explanation pool as a fallback option if LLM cannot generate a reasonable fix given the students’ explanation. At a higher level, various prior work on appropriate reliance on AIs can be utilized to guardrail training with imperfect AIs, e.g., the use of analogies, confidence calibrations, etc. [10, 39], and exploring their adoption in the education domain will be promising.

9.2 Explicit Practice on Hypothesis Construction (R.2): Skills to Offload

Our user study students indeed appreciated offloading code completion to LLM. However, empirically, we found that tracing code and simulating inaccurate input in a program remain challenging to novices, when they are trying to formulate accurate hypotheses about buggy code. We plan to explicitly address this issue, perhaps by guiding students through the code via conversations or by incorporating external libraries that enable code tracing.

Thinking more broadly about what skills to train students on (and what to offload) in the era of LLM/AI, we see two possibilities of LLM integration: (1) A less invasive usage, where LLMs are integrated into existing education schema such that students are trained to be programmers in standard ways, just with LLMs support; (2) A more radical framing, where students are trained for human-AI collaboration, e.g., through the context of human-LLM pair programming [59]. Our ad-hoc discussions with CS instructors at our university showed that they tend to prefer the former, arguing that most skills will still be necessary even in the era of co-programming with LLMs (though they acknowledged the role of LLMs in the future of programming).

We see debugging training (and HYPOCOMPASS) as a sweet spot balancing both ends: it trains students (1) on an existing programming skill (2) that is of increasing importance with the presence of LLMs. Still, more explorations on when students need to be trained in what way can be quite interesting. On the one hand, we should identify the *bare minimum* low-level programming skills to make using LLMs feasible (e.g., understanding basic syntax). On the other hand, we should also identify the *bare minimum* human-AI collaboration skills that need to be inserted into the CS curriculum. For example, task re-framing seems to be an important yet rarely grasped skill for human-LLM collaboration. In Section 5.1, we note that LLM performance varies greatly *purely based on how a task is defined*, e.g., removing competing tasks through *partial code translation* is an intuition that most users do not have. Re-framing skill has been widely applied in design contexts [92], and transferring such skill into CS training might be low-cost yet beneficial.

9.3 Scaffolded and Engaging Learning (R.3): Towards Better Personalization

While novices highly appreciated detailed hints (such as test case categories and explanation pools) and structured interactions, more advanced students could find them distracting. It is interesting to further explore incorporating students’ learning progress into the system for enhanced personalization. For instance, we could implement faded scaffolding, where we gradually remove hints as the training progresses, with the speed customized based on detected student proficiency (e.g., through clickstream logs). We can also consider prioritizing addressing buggy code that has proven

to be challenging for a particular student, based on their previous attempts to select the explanation for a similar buggy solution.

One of the most intriguing areas for improvement lies in the generation of explanations. While we assessed the correctness of explanations on a binary basis, there exist numerous finer dimensions for customization from a learning perspective, including levels of granularity, the choice between multi-turn and single-turn conversation, and more. One intuitive improvement would be to generate standard explanations, then use the LLM to rewrite and transfer for various desired attributes (e.g., make the explanation shorter, post in the form of a question, etc.) The relevance of distractors is also customizable — we can ground the generation of distractor explanations based on students’ prior incorrect explanations, thereby allowing them to practice common misconceptions.

More broadly, LLMs seem a promising source for multi-faceted personalization. LLMs are known to generate data according to attributes in prompts [94], and we plan to develop taxonomies on attributes-of-interest for making the scaffolds more personalized.

9.4 Efficient Material Generation (R.4): What Minimizes Instructor/Student Effort?

Though most students found the HYPOCOMPASS engaging, some found having to interact with an interface not straightforward and preferred to practice debugging in their natural coding environments (e.g., IDEs like VSCode). Some instructors also expressed concerns on the deployment and maintenance cost (Section 7). Essentially, this contradicting feedback highlights that the term “minimal effort” is subjective.

To maximize utility to different user groups, it might be interesting to modularize different components in HYPOCOMPASS. For example, the instructor in Section 7.4 who prefer to distribute training materials as static handouts can rely entirely on the material generation module, whereas instructors who want to experiment with the training setup but are skeptical about LLM involvement can pair the teachable agent interface with buggy codes generated by human students, rather than LLMs.

In this work, we primarily evaluated *whether the full HYPOCOMPASS can bring learning gains* through a small in-lab experiment, which serves as a prerequisite for more extensive classroom deployment. For next step, we plan to perform more ablation studies to disentangle the gains from different HYPOCOMPASS components. For example, we will compare HYPOCOMPASS, with LLM-generated materials distributed as handouts, and with existing course materials (handouts with descriptive tips on debugging and exercises). Our preliminary experiments show that the LLM-generated materials also bring marginal learning gain, though not as much as HYPOCOMPASS. With a larger-scale controlled experiment, we hope to help instructors select the best version that fits their needs.

ACKNOWLEDGMENTS

Thanks to all the participants for the pilot, interview, and user study in this work. Thanks to Kelly Rivers, Michael Taylor, and Michael Hilton for instructor insights and design advice.

REFERENCES

- [1] 2014. Why Don't Schools Teach Debugging? <https://news.ycombinator.com/item?id=7215870>. <https://news.ycombinator.com/item?id=7215870> Accessed: 2023-9-8.
- [2] Michael Agyemang Adarkwah, Chen Ying, Muhammad Yasir Mustafa, and Ronghuai Huang. 2023. Prediction of Learner Information-Seeking Behavior and Classroom Engagement in the Advent of ChatGPT. In *International Conference on Smart Learning Environments*. Springer, 117–126.
- [3] Wolfgang Ahrendt, Richard Bubel, and Reiner Hähnle. 2009. Integrated and Tool-Supported Teaching of Testing, Debugging, and Verification. In *Teaching Formal Methods*. Springer Berlin Heidelberg, 125–143. https://doi.org/10.1007/978-3-642-04912-5_9
- [4] Miltiadis Allamanis, Henry Jackson-Flux, and Marc Brockschmidt. 2021. Self-supervised bug detection and repair. *Advances in Neural Information Processing Systems* 34 (2021), 27865–27876.
- [5] Miltiadis Allamanis, Henry Jackson-Flux, and Marc Brockschmidt. 2021. Self-Supervised Bug Detection and Repair. (May 2021). arXiv:2105.12787 [cs.LG] <http://arxiv.org/abs/2105.12787>
- [6] K Araki, Z Furukawa, and J Cheng. 1991. A general framework for debugging. *IEEE Softw.* 8, 3 (May 1991), 14–20. <https://doi.org/10.1109/52.88939>
- [7] Pasquale Ardimento, Mario Luca Bernardi, Marta Cimitile, and Giuseppe De Ruvo. 2019. Reusing Bugged Source Code to Support Novice Programmers in Debugging Tasks. *ACM Trans. Comput. Educ.* 20, 1 (Nov. 2019), 1–24. <https://doi.org/10.1145/3355616>
- [8] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732* (2021).
- [9] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, H Michalewski, David Dohan, Ellen Jiang, Carrie J Cai, Michael Terry, Quoc V Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *ArXiv* (2021). <https://www.semanticscholar.org/paper/a38e0f993e4805ba8a9beae4c275c91ffec01df>
- [10] Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. 2021. Does the whole exceed its parts? the effect of ai explanations on complementary team performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. 1–16.
- [11] Brett A Becker, Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, James Prather, and Eddie Antonio Santos. 2023. Programming is hard-or at least it used to be: Educational opportunities and challenges of ai code generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 500–506.
- [12] Christian Bird, Denae Ford, Thomas Zimmermann, Nicole Forsgren, Eirini Kalliamvakou, Travis Lowdermilk, and Idan Gazit. 2022. Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue* 20, 6 (2022), 35–57.
- [13] Kristen Blair, Daniel I. Schwartz, Gautam Biswas, and Krittaya Leelawong. 2007. Pedagogical Agents for Learning by Teaching: Teachable Agents. *Educ. Technol. Res. Dev.* 47, 1 (2007), 56–61. <http://www.jstor.org/stable/44429380>
- [14] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. 2023. Sparks of Artificial General Intelligence: Early experiments with GPT-4. (March 2023). arXiv:2303.12712 [cs.CL] <http://arxiv.org/abs/2303.12712>
- [15] Guillermo Campitelli and Fernand Gobet. 2011. Deliberate Practice: Necessary But Not Sufficient. *Curr. Dir. Psychol. Sci.* 20, 5 (Oct. 2011), 280–285. <https://doi.org/10.1177/0963721411421922>
- [16] Elizabeth Emily Carter. 2014. *An Intelligent Debugging Tutor For Novice Computer Science Students*. Ph.D. Dissertation. Lehigh University. <https://core.ac.uk/download/pdf/228636792.pdf>
- [17] Venera-Mihaela Cojocariu and Gabriel Mareş. 2022. Academic Integrity in the Technology-Driven Education Era. In *Ethical Use of Information Technology in Higher Education*, Liliana Măță (Ed.). Springer Singapore, Singapore, 1–16. https://doi.org/10.1007/978-981-16-1951-9_1
- [18] GitHub Copilot. 2021. Your AI pair programmer.
- [19] Wei Dai, Jionghao Lin, Flora Jin, Tongguang Li, Yi-Shan Tsai, Dragan Gasevic, and Guanliang Chen. 2023. Can Large Language Models Provide Feedback to Students? A Case Study on ChatGPT. (April 2023). <https://doi.org/10.35542/osf.io/hcgzj>
- [20] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, Zhen Ming, and Jiang. 2022. GitHub Copilot AI pair programmer: Asset or Liability? *ArXiv* (2022). <https://doi.org/10.48550/ARXIV.2206.15331>
- [21] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto ON, Canada) (SIGCSE 2023). Association for Computing Machinery, New York, NY, USA, 1136–1142. <https://doi.org/10.1145/3545945.3569823>
- [22] Chetan Desai, David S Janzen, and John Clements. 2009. Implications of integrating test-driven development into CS1/CS2 curricula. *SIGCSE Bull.* 41, 1 (March 2009), 148–152. <https://doi.org/10.1145/1539024.1508921>
- [23] Thomas Dohmke. 2023. GitHub Copilot X: The AI-powered developer experience. <https://github.blog/2023-03-22-github-copilot-x-the-ai-powered-developer-experience/>. <https://github.blog/2023-03-22-github-copilot-x-the-ai-powered-developer-experience/> Accessed: 2023-9-5.
- [24] Gabrielle E Ecanow. 2023. *Debug Tutor: Automated Deliberate Debugging Practice for Undergraduate Programmers*. Ph.D. Dissertation. Massachusetts Institute of Technology. <https://dspace.mit.edu/handle/1721.1/151452?show=full>
- [25] Stephen H Edwards and Zalia Shams. 2014. Comparing test quality measures for assessing student-written tests. In *Companion Proceedings of the 36th International Conference on Software Engineering* (Hyderabad, India) (ICSE Companion 2014). Association for Computing Machinery, New York, NY, USA, 354–363. <https://doi.org/10.1145/2591062.2591164>
- [26] Stephen H Edwards and Zalia Shams. 2014. Do student programmers all tend to write the same software tests?. In *Proceedings of the 2014 conference on Innovation & technology in computer science education* (Uppsala, Sweden) (ITiCSE '14). Association for Computing Machinery, New York, NY, USA, 171–176. <https://doi.org/10.1145/2591708.2591757>
- [27] Anders Ericsson and Robert Pool. 2016. *Peak: Secrets from the new science of expertise*. Random House. https://durmonski.com/wp-content/uploads/2022/03/Peak-by-Anders-Ericsson_Summary.pdf
- [28] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.
- [29] Zhiyu Fan, Xiang Gao, Abhik Roychoudhury, and Shin Hwei Tan. 2022. Automated Repair of Programs from Large Language Models. *arXiv preprint arXiv:2205.10583* (2022).
- [30] Zhiyu Fan, Xiang Gao, Abhik Roychoudhury, and Shin Hwei Tan. 2022. Improving automatically generated code from Codex via Automated Program Repair. *ArXiv* (2022). <https://doi.org/10.48550/ARXIV.2205.10583>
- [31] James Finnie-Ansley, Paul Denny, Brett A Becker, Andrew Luxton-Reilly, and James Prather. 2022. The robots are coming: Exploring the implications of openai codex on introductory programming. In *Proceedings of the 24th Australasian Computing Education Conference*. 10–19.
- [32] Sue Fitzgerald, Gary Lewandowski, Renée McCauley, Laurie Murphy, Beth Simon, Lynda Thomas, and Carol Zander. 2008. Debugging: finding, fixing and failing, a multi-institutional study of novice debuggers. *Comput. Sci. Educ.* 18, 2 (June 2008), 93–116. <https://doi.org/10.1080/08993400802114508>
- [33] Sue Fitzgerald, Gary Lewandowski, Renée McCauley, Laurie Murphy, Beth Simon, Lynda Thomas, and Carol Zander. 2008. Debugging: finding, fixing and failing, a multi-institutional study of novice debuggers. *Computer Science Education* 18, 2 (2008), 93–116.
- [34] Sue Fitzgerald, Renée McCauley, Brian Hanks, Laurie Murphy, Beth Simon, and Carol Zander. 2010. Debugging From the Student Perspective. *IEEE Trans. Educ.* 53, 3 (Aug. 2010), 390–396. <https://doi.org/10.1109/TE.2009.2025266>
- [35] Luciano Floridi. 2023. AI as agency without intelligence: on ChatGPT, large language models, and other generative models. *Philosophy & Technology* 36, 1 (2023), 15.
- [36] Deep Ganguli, Danny Hernandez, Liane Lovitt, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova Dassarma, Dawn Drain, Nelson Elhage, et al. 2022. Predictability and surprise in large generative models. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. 1747–1764.
- [37] Philip Guo. 2021. Ten million users and ten years later: Python tutor's design guidelines for building scalable and sustainable research software in academia. In *The 34th Annual ACM Symposium on User Interface Software and Technology* (Virtual Event USA). ACM, New York, NY, USA. <https://doi.org/10.1145/3472749.3474819>
- [38] Patrick Haluptzok, Matthew Bowers, and Adam Tauman Kalai. 2022. Language models can teach themselves to program better. *ArXiv* (2022). <https://doi.org/10.48550/ARXIV.2207.14502>
- [39] Gaole He and Ujwal Gadiraju. 2022. Walking on Eggshells: Using Analogies to Promote Appropriate Reliance in Human-AI Decision Making. In *Proceedings of the Workshop on Trust and Reliance on AI-Human Teams at the ACM Conference on Human Factors in Computing Systems (CHI'22)*.
- [40] Dan Hendrycks, Nicholas Carlini, John Schulman, and Jacob Steinhardt. 2021. Unsolved problems in ml safety. *arXiv preprint arXiv:2109.13916* (2021).
- [41] Matthew Hertz. 2010. What Do "CS1" and "CS2" Mean? Investigating Differences in the Early Courses. In *Proceedings of the 41st ACM Technical Symposium on Computer Science Education* (Milwaukee, Wisconsin, USA) (SIGCSE '10). Association for Computing Machinery, New York, NY, USA, 199–203. <https://doi.org/10.1145/1734263.1734335>
- [42] Yang Hu, Umair Z Ahmed, Sergey Mechtav, Ben Leong, and Abhik Roychoudhury. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 388–398. <https://doi.org/10.1109/ASE.2019.00044>

- [43] Ville Isomöttönen and Vesa Lappalainen. 2012. CSI with games and an emphasis on TDD and unit testing: piling a trend upon a trend. *ACM Inroads* 3, 3 (Sept. 2012), 62–68. <https://doi.org/10.1145/2339055.2339073>
- [44] Maria Kallia. 2023. The Search for Meaning: Inferential Strategic Reading Comprehension in Programming. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1 (ICER '23 V1)* (Chicago, IL, United States). ACM. <https://eprints.gla.ac.uk/304509/>
- [45] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2022. Large Language Models are few-shot testers: Exploring LLM-based general bug reproduction. *ArXiv* (2022). <https://doi.org/10.48550/ARXIV.2209.11515>
- [46] I R Katz and J R Anderson. 1987. Debugging: An analysis of bug-location strategies. *Human-Computer Interaction* (1987). https://www.tandfonline.com/doi/abs/10.1207/s15327051hci0304_2?casa_token=O3HmaPI_gUMAAAAA: Dw7niiGqcpTclx4iIGHYVmhRL9D_uWb-Uku-xLcGoZPSze4JVaubD5o2iXP3iitFzWdytFqFdRF
- [47] David Klahr and Sharon Mccoy Carver. 1988. Cognitive objectives in a LOGO debugging curriculum: Instruction, learning, and transfer. *Cogn. Psychol.* 20, 3 (July 1988), 362–404. [https://doi.org/10.1016/0010-0285\(88\)90004-7](https://doi.org/10.1016/0010-0285(88)90004-7)
- [48] Amy J Ko and Brad A Myers. 2008. Debugging reinvented: asking and answering why and why not questions about program behavior. In *Proceedings of the 30th international conference on Software engineering* (Leipzig, Germany) (ICSE '08). Association for Computing Machinery, New York, NY, USA, 301–310. <https://doi.org/10.1145/1368088.1368130>
- [49] Kenneth R Koedinger, Julie L Booth, and David Klahr. 2013. Instructional Complexity and the Science to Constrain It. *Science* 342, 6161 (2013), 935–937. <https://doi.org/10.1126/science.1238056> <https://www.science.org/doi/pdf/10.1126/science.1238056>
- [50] Arto Koutchme, charles and Sarsa, Sami and Leinonen, Juho and Haaranen, Lassi and Hellas. 2023. Evaluating Distance Measures for Program Repair. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1 (ICER '23 V1)* (Chicago, IL, USA). ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3568813.3600130>
- [51] Shuvendu K Lahiri, Aaditya Naik, Georgios Sakkas, Piali Choudhury, Curtis von Veh, Madanlal Musuvathi, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. 2022. Interactive code generation via test-driven user-intent formalization. *arXiv preprint arXiv:2208.05950* (2022).
- [52] Shuvendu K Lahiri, Aaditya Naik, Georgios Sakkas, Piali Choudhury, Curtis von Veh, Madanlal Musuvathi, Jeevana Priya Inala, Chenglong Wang, and Jianfeng Gao. 2022. Interactive code generation via test-driven user-intent formalization. *ArXiv* (2022). <https://doi.org/10.48550/ARXIV.2208.05950>
- [53] V C S Lee, Y T Yu, C M Tang, T L Wong, and C K Poon. 2018. ViDA: A virtual debugging advisor for supporting learning in computer programming courses. *J. Comput. Assist. Learn.* 34, 3 (June 2018), 243–258. <https://doi.org/10.1111/jcal.12238>
- [54] Zongjie Li, Chaozheng Wang, Zhibo Liu, Haoxuan Wang, Shuai Wang, and Cuiyun Gao. 2022. CCTEST: Testing and repairing code completion systems. *ArXiv* (2022). <https://doi.org/10.48550/ARXIV.2208.08289>
- [55] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2021. Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. (July 2021). [arXiv:2107.13586 \[cs.CL\]](https://arxiv.org/abs/2107.13586) <http://arxiv.org/abs/2107.13586>
- [56] Alena Lukášová. 1979. Hierarchical agglomerative clustering procedure. *Pattern Recognition* 11, 5-6 (1979), 365–381.
- [57] Andrew Luxton-Reilly, Emma McMillan, Elizabeth Stevenson, Ewan Tempero, and Paul Denny. 2018. Ladebug: an online tool to help novice programmers improve their debugging skills. In *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education (Larnaca, Cyprus) (ITiCSE 2018)*. Association for Computing Machinery, New York, NY, USA, 159–164. <https://doi.org/10.1145/3197091.3197098>
- [58] Andrew Luxton-Reilly, Simon, Ibrahim Albluwi, Brett A Becker, Michail Gianakos, Amruth N Kumar, Linda Ott, James Paterson, Michael James Scott, Judy Sheard, and Claudia Szabo. 2018. Introductory programming: a systematic literature review. In *Proceedings Companion of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education (Larnaca, Cyprus) (ITiCSE 2018 Companion)*. Association for Computing Machinery, New York, NY, USA, 55–106. <https://doi.org/10.1145/3293881.3295779>
- [59] Qianou Ma, Tongshuang Wu, and Kenneth Koedinger. 2023. Is AI the better programming partner? Human-Human Pair Programming vs. Human-AI pAIR Programming. (June 2023). [arXiv:2306.05153 \[cs.HC\]](https://arxiv.org/abs/2306.05153) <http://arxiv.org/abs/2306.05153>
- [60] Stephen MacNeil, Andrew Tran, Dan Mogil, Seth Bernstein, Erin Ross, and Ziheng Huang. 2022. Generating diverse code explanations using the gpt-3 large language model. In *Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 2*. 37–39.
- [61] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651* (2023).
- [62] Julia M Markel, Steven G Opferman, James A Landay, and Chris Piech. 2023. GPTeach: Interactive TA Training with GPT-based Students. In *Proceedings of the Tenth ACM Conference on Learning @ Scale (Copenhagen, Denmark) (L@S '23)*. Association for Computing Machinery, New York, NY, USA, 226–236. <https://doi.org/10.1145/3573051.3593393>
- [63] Samiha Marwan, Joseph Jay Williams, and Thomas Price. 2019. An evaluation of the impact of automated programming hints on performance and learning. In *Proceedings of the 2019 ACM Conference on International Computing Education Research*. 61–70.
- [64] Noboru Matsuda, Evelyn Yarzebinski, Victoria Keiser, Rohan Raizada, William W Cohen, Gabriel J Stylianides, and Kenneth R Koedinger. 2013. Cognitive anatomy of tutor learning: Lessons learned with SimStudent. *J. Educ. Psychol.* 105, 4 (Nov. 2013), 1152–1163. <https://doi.org/10.1037/a0031955>
- [65] Renee McCauley, Sue Fitzgerald, Gary Lewandowski, Laurie Murphy, Beth Simon, Lynda Thomas, and Carol Zander. 2008. Debugging: A Review of the Literature from an Educational Perspective. *Computer Science Education* 18, 2 (June 2008), 67–92. <https://doi.org/10.1080/08993400802114581>
- [66] Renee McCauley, Sue Fitzgerald, Gary Lewandowski, Laurie Murphy, Beth Simon, Lynda Thomas, and Carol Zander. 2008. Debugging: a review of the literature from an educational perspective. *Computer Science Education* 18, 2 (2008), 67–92.
- [67] Tilman Michaeli and Ralf Romeike. 2019. Improving Debugging Skills in the Classroom: The Effects of Teaching a Systematic Debugging Process. In *Proceedings of the 14th Workshop in Primary and Secondary Computing Education (Glasgow, Scotland, UK) (WiPSCPE '19, Article 15)*. Association for Computing Machinery, New York, NY, USA, 1–7. <https://doi.org/10.1145/3361721.3361724>
- [68] Melanie Mitchell and David C Krakauer. 2023. The debate over understanding in AI's large language models. *Proceedings of the National Academy of Sciences* 120, 13 (2023), e2215907120.
- [69] Hussein Moazzam, Gagan Bansal, Adam Fournay, and Eric Horvitz. 2022. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. *arXiv preprint arXiv:2210.14306* (2022).
- [70] Ella Neeman, Roei Aharoni, Or Honovich, Leshem Choshen, Idan Szepkator, and Omri Abend. 2022. Disentangling parametric and contextual knowledge with counterfactual question answering. *arXiv preprint arXiv:2211.05655* (2022).
- [71] Amy Ogan, Samantha Finkelstein, Elijah Mayfield, Claudia D'Adamo, Noboru Matsuda, and Justine Cassell. 2012. "Oh dear stacy!": social interaction, elaboration, and learning with teachable agents. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Austin, Texas, USA) (CHI '12). Association for Computing Machinery, New York, NY, USA, 39–48. <https://doi.org/10.1145/2207676.2207684>
- [72] Zachary A Pardos and Shreya Bhandari. 2023. Learning gain differences between ChatGPT and human tutor generated algebra hints. (Feb. 2023). [arXiv:2302.06871 \[cs.CY\]](https://arxiv.org/abs/2302.06871) <http://arxiv.org/abs/2302.06871>
- [73] Michael Perscheid, Benjamin Siegmund, Marcel Taeumel, and Robert Hirschfeld. 2017. Studying the advancement in debugging practice of professional software developers. *Software Quality Journal* 25, 1 (March 2017), 83–110. <https://doi.org/10.1007/s11219-015-9294-2>
- [74] Tung Phung, Victor-Alexandru Pădurean, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generative AI for Programming Education: Benchmarking ChatGPT, GPT-4, and Human Tutors. (June 2023). [arXiv:2306.17156 \[cs.CY\]](https://arxiv.org/abs/2306.17156) <http://arxiv.org/abs/2306.17156>
- [75] James Prather, Brent N Reeves, Paul Denny, Brett A Becker, Juho Leinonen, Andrew Luxton-Reilly, Garrett Powell, James Finnie-Ansley, and Eddie Antonio Santos. 2023. "It's Weird That it Knows What I Want": Usability and Interactions with Copilot for Novice Programmers. *arXiv preprint arXiv:2304.02491* (2023).
- [76] Arun Raman and Viraj Kumar. 2022. Programming Pedagogy and Assessment in the Era of AI/ML: A Position Paper. In *Proceedings of the 15th Annual ACM India Compute Conference (Jaipur, India) (COMPUTE '22)*. Association for Computing Machinery, New York, NY, USA, 29–34. <https://doi.org/10.1145/3561833.3561843>
- [77] Eric Roberts, John Lilly, and Bryan Rollins. 1995. Using undergraduates as teaching assistants in introductory programming courses: An update on the Stanford experience. In *Proceedings of the twenty-sixth SIGCSE technical symposium on Computer science education*. 48–52.
- [78] Jekaterina Rogaten, Bart Rienties, Rhona Sharpe, Simon Cross, Denise Whitelock, Simon Lygo-Baker, and Allison Littlejohn. 2019. Reviewing affective, behavioural and cognitive learning gains in higher education. *Assessment & Evaluation in Higher Education* 44, 3 (April 2019), 321–337. <https://doi.org/10.1080/02602938.2018.1504277>
- [79] Steven I Ross, Fernando Martinez, Stephanie Houde, Michael Muller, and Justin D Weisz. 2023. The programmer's assistant: Conversational interaction with a large language model for software development. In *Proceedings of the 28th International Conference on Intelligent User Interfaces*. 491–514.
- [80] Jaromir Savelka, Arav Agarwal, Marshall An, Chris Bogart, and Majd Sakr. 2023. Thrilled by Your Progress! Large Language Models (GPT-4) No Longer Struggle to Pass Assessments in Higher Education Programming Courses. (June 2023). [arXiv:2306.10073 \[cs.CY\]](https://arxiv.org/abs/2306.10073) <http://arxiv.org/abs/2306.10073>

- [81] Logan Stapleton, Jordan Taylor, Sarah Fox, Tongshuang Wu, and Haiyi Zhu. 2023. Seeing Seeds Beyond Weeds: Green Teaming Generative AI for Beneficial Uses. *arXiv preprint arXiv:2306.03097* (2023).
- [82] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [83] Jan Philip Wahle, Terry Ruas, Norman Meuschke, and Bela Gipp. 2021. Are neural language models good plagiarists? A benchmark for neural paraphrase detection. In *2021 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*. IEEE, 226–229.
- [84] Jacqueline Whalley, Amber Settle, and Andrew Luxton-Reilly. 2021. Analysis of a Process for Introductory Debugging. In *Australasian Computing Education Conference (Virtual, SA, Australia) (ACE '21)*. Association for Computing Machinery, New York, NY, USA, 11–20. <https://doi.org/10.1145/3441636.3442300>
- [85] Jacqueline Whalley, Amber Settle, and Andrew Luxton-Reilly. 2023. A Think-aloud Study of Novice Debugging. *ACM Trans. Comput. Educ.* (March 2023). <https://doi.org/10.1145/3589004>
- [86] Grant P Wiggins and Jay McTighe. 2005. *Understanding by Design*. ASCD. <https://play.google.com/store/books/details?id=N2EfKlyUN4QC>
- [87] Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) (*CHI '22, Article 385*). Association for Computing Machinery, New York, NY, USA, 1–22. <https://doi.org/10.1145/3491102.3517582>
- [88] Tongshuang Wu, Haiyi Zhu, Maya Albayrak, Alexis Axon, Amanda Bertsch, Wenxing Deng, Ziqi Ding, Bill Guo, Sireesh Gururaja, Tzu-Sheng Kuo, et al. 2023. LLMs as Workers in Human-Computational Algorithms? Replicating Crowdsourcing Pipelines with LLMs. *arXiv preprint arXiv:2307.10168* (2023).
- [89] Benjamin Xie, Dastyni Loksa, Greg L Nelson, Matthew J Davidson, Dongsheng Dong, Harrison Kwik, Alex Hui Tan, Leanne Hwa, Min Li, and Amy J Ko. 2019. A theory of instruction for introductory programming skills. *Computer Science Education* 29, 2-3 (July 2019), 205–253. <https://doi.org/10.1080/08993408.2019.1565235>
- [90] Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2023. Adaptive Chameleon or Stubborn Sloth: Unraveling the Behavior of Large Language Models in Knowledge Clashes. *arXiv:2305.13300* [cs.CL]
- [91] Shaochun Xu and V Rajlich. 2004. Cognitive process during program debugging. In *Proceedings of the Third IEEE International Conference on Cognitive Informatics, 2004*. (Victoria, BC, Canada). IEEE, 176–182. <https://doi.org/10.1109/COGINF.2004.1327473>
- [92] Qian Yang, Justin Cranshaw, Saleema Amershi, Shamsi T Iqbal, and Jaime Teevan. 2019. Sketching nlp: A case study of exploring the right things to design with language intelligence. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–12.
- [93] Burak Yetiştir, İşik Özsoy, and Eray Tüzün. 2022. Assessing the Quality of GitHub Copilot’s Code Generation. In *18th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE '22)*. <https://doi.org/10.1145/3558489.3559072>
- [94] Yue Yu, Yuchen Zhuang, Jieyu Zhang, Yu Meng, Alexander Ratner, Ranjay Krishna, Jiaming Shen, and Chao Zhang. 2023. Large Language Model as Attributed Training Data Generator: A Tale of Diversity and Bias. *arXiv preprint arXiv:2306.15895* (2023).
- [95] Andreas Zeller. 2009. *Why programs fail: A guide to systematic debugging* (2 ed.). Morgan Kaufmann, Oxford, England. <https://doi.org/10.1016/b978-0-12-374515-6.x0000-7>
- [96] Shuyan Zhou, Uri Alon, Frank F Xu, Zhengbao Jiang, and Graham Neubig. 2022. DocCoder: Generating code by retrieving and reading docs. *ArXiv* (2022). <https://doi.org/10.48550/ARXIV.2207.05987>
- [97] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.
- [98] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming* (San Diego CA USA). ACM, New York, NY, USA. <https://doi.org/10.1145/3520312.3534864>

A LLM EXPERIMENT MATERIALS

Table 3: Description, number of generated bugs, and number of final fixed codes for problems used in LLM experiment

Problem	Description	# Bugs	# Fixes
first_num_greater_than	Write a Python function <code>first_num_greater_than(numbers_list, key)</code> that takes a list of integers (<code>numbers_list</code>) and an integer key (<code>key</code>), and returns the first number in the list that is greater than the key. If there is no number greater than the key, then you should return <code>None</code> .	30	36
remove_extras	Function <code>remove_extras(lst)</code> takes in a list of integers and returns a new list with the first occurrence of each element, which is the same as <code>lst</code> but with all repeated occurrences of any element removed.	23	31
num_smaller	Function <code>num_smaller(seq, x)</code> takes in an integer <code>x</code> and a sorted integer sequence <code>seq</code> , and returns the number of elements in <code>seq</code> that is strictly smaller than <code>x</code> .	18	24
sort_age	We represent a person using a tuple (<code><gender></code> , <code><age></code>). Given a list of people, write a function <code>sort_age</code> that returns a list in an order such that the older people are at the front of the list. You may assume that no two members in the list of people are of the same age.	28	42
top_k	Write a function <code>top_k</code> that takes a list of integers as the input and returns the greatest <code>k</code> number of values as a list, with its elements sorted in descending order. You may use any sorting algorithm, but you are not allowed to use the Python function <code>sort</code> and <code>sorted</code> .	28	42
swap_keys_values	Write a function <code>swap_keys_values</code> that takes in a dictionary, and returns a new dictionary with the keys and values swapped.	18	20
Total		145	195

Table 4: Human annotation tasks and success rate definitions for each type of material generated in LLM experiment

Material	Success Rate Definition	Human Label Tasks
Buggy code	$\frac{\# \text{ behaviorally distinct buggy code}}{\# \text{ non-identical buggy code}}$	remove extra comments e.g., <code># the operator should actually be ></code>
Bug explanation and fix instruction	$\frac{\# \text{ correct expl \& fix}}{\# \text{ total buggy code}}$	<code>multibug_correct</code> : T/F (does this code have meaningful and isolate fixes for the different bugs it specify), NA (single bug) <code>expl_correct</code> : T/F (does the explanation accurately reflect the bug in code) <code>fix_correct</code> : T/F (does the fix correctly and minimally fix the bug in code) <code>edited_expl_fix</code> : edited for next step if needed { <code>expl:...</code> ; <code>fix:...</code> }
Fixed Code	$\frac{\# \text{ accurately fixed code}}{\# \text{ total buggy code}}$	<code>fix_round1_correct</code> : T/F & <code>edited_round1_fix</code> (does the code snippet accurately reflect the instruction without over or under-fix, edit if needed) <code>fix_round2_correct</code> : T/F & <code>edited_fixed_code</code> (does the fixed code accurately reflect the instruction without over or under-fix, edit if needed)
Test case description hint	$\frac{\# \text{ correct test case hint}}{\# \text{ total test case}}$	<code>tc_hint_correct</code> : T/F & <code>edited_tc_hint</code> (does this hint accurately describe what the test case is doing, edit if needed) Edited <code>tc hint</code>
Test case category	$\frac{\# \text{ correct test category}}{\# \text{ total test category}}$	<code>tc_category_correct</code> : T/F & <code>edited_tc_category</code> (is this category reasonable to test for this problem, edit if needed)